

中图分类号：TP317

论文编号：100006SY2306142

北京航空航天大学
硕士学位论文

面向云边端协同的跨域分布式训练通信优化研究

作者姓名	李云潼
学科专业	计算机体系结构
指导教师	肖利民 教授
培养学院	计算机学院

Research on Communication Optimization for Distributed Training in Cross-Domain Multi-Data Center Scenarios

A Dissertation Submitted for the Degree of Master of Engineering

Candidate: Yuntong Li

Supervisor: Prof. Limin Xiao

School of Computer Science and Engineering
Beihang University, Beijing, China

中图分类号：TP317

论文编号：100006SY2306142

硕 士 学 位 论 文

面向云边端协同的跨域分布式训练通信优化研究

作者姓名	李云潼	申请学位级别	工学硕士
指导教师姓名	肖利民	职 称	教授
学科专业	计算机体系结构	研究方向	模型分布式训练
一级学科	计算机科学与技术	学科方向	计算机体系结构
学习时间自	2021 年 09 月 01 日	至	2026 年 06 月 30 日
论文提交日期	2026 年 06 月 10 日	论文答辩日期	2026 年 06 月 10 日
学位授予单位	北京航空航天大学	学位授予日期	年 月 日

关于学位论文的独创性声明

本人郑重声明：所呈交的论文是本人在指导教师指导下独立进行研究工作所取得的成果，论文中有关资料和数据是实事求是的。尽我所知，除文中已经加以标注和致谢外，本论文不包含其他人已经发表或撰写过的研究成果，也不包含本人或他人为获得北京航空航天大学或其它教育机构的学位或学历证书而使用过的材料。与我一同工作的同志对研究所做的任何贡献均已在论文中作出了明确的说明。

若有不实之处，本人愿意承担相关法律责任。

学位论文作者签名：_____

日期：____年__月__日

学位论文使用授权书

本人完全同意北京航空航天大学有权使用本学位论文（包括但不限于其印刷版和电子版），使用方式包括但不限于：保留学位论文，按规定向国家有关部门（机构）送交学位论文，以学术交流为目的赠送和交换学位论文，允许学位论文被查阅、借阅和复印，将学位论文的全部或部分内容编入有关数据库进行检索，采用影印、缩印或其他复制手段保存学位论文。

保密学位论文在解密后的使用授权同上。

学位论文作者签名：_____

日期：____年__月__日

指导教师签名：_____

日期：____年__月__日

摘 要

在大语言模型训练中，分布式训练技术已成为支撑大规模模型训练的核心基础。随着训练数据与算力资源从“单一数据中心”走向“云-边-端协同”的跨域形态：数据分布在云端数据湖、边缘微型数据中心与端侧设备，算力分散在中心云 GPU 集群、边缘 GPU/CPU 节点与部分端侧加速器。训练系统面临更强的异构性：域内链路（如同机 NVLink/PCIe、同域 RDMA）高带宽低时延，而跨域链路（边缘到云、边缘到边缘、端到边缘的蜂窝/宽带接入）往往带宽更低、RTT 更高且抖动更大。由此产生的梯度同步通信开销更容易进入迭代关键路径，成为制约云边端协同训练吞吐与时延的关键瓶颈。

针对上述挑战，本文以“跨域（Cloud - Edge - Device）分布式训练通信优化”为目标，从通信量、集合通信调度、计算-通信重叠三个层面开展系统性研究。本文主要工作如下：

（1）通信数据量优化：提出 k -bit 随机舍入量化分布式优化方法，支持 $b \in \{1, 2, 4, 8, 16\}$ 的可配置位宽。在受限链路条件下，通信数据量最高可降低至原有的 $1/32$ ，并可在压缩率与收敛稳定性之间进行可调折中；同时设计与之配套的量化同步与聚合机制，实现对低比特量化张量的高效通信支持。

（2）集合通信调度优化：系统分析跨数据中心场景下的 All-Reduce 通信过程，针对 NCCL 的 CollNet 通信机制进行改进，提出多层流水线通信调度策略，充分利用集群间和集群内的异构带宽资源，显著加速通信过程。

（3）计算-通信重叠优化：量化分析混合并行策略下的计算-通信重叠程度，对比跨数据中心通信与数据中心内部通信的重叠差异，通过重新设计混合并行模式下的梯度同步机制，有效提升计算-通信掩盖程度。

关键词：分布式训练，云边端协同，跨域通信，通信优化，大语言模型

Abstract

Distributed training has become the fundamental infrastructure for large language model training. As training data and compute resources evolve from a single data center to cloud-edge-device collaboration—where data resides across central clouds, edge micro-data centers, and end devices, and compute spans cloud GPU clusters and heterogeneous edge nodes—training systems face stronger cross-domain heterogeneity: intra-domain links (e.g., NVLink/PCIe, RDMA) are high-bandwidth and low-latency, while cross-domain links (edge-to-cloud, edge-to-edge, and device-to-edge access networks) are typically bandwidth-limited with higher RTT and larger jitter. As a result, gradient synchronization is more likely to fall onto the critical path and become a key bottleneck for end-to-end throughput and latency.

To address these challenges, this thesis targets communication optimization for cross-domain (Cloud – Edge – Device) distributed training. Cross-data-center training is treated as a representative, strongly-constrained instance of cloud-edge / edge-edge interconnects, capturing the key characteristics of low bandwidth and high RTT. The main contributions are as follows:

(1) **Communication Volume Optimization:** We propose a k -bit stochastic-rounding distributed optimization method with configurable bit widths ($b \in \{1, 2, 4, 8, 16\}$). Under constrained links, it reduces communication volume by up to 32x (1/32 of the original size) while enabling a tunable trade-off between compression ratio and convergence stability; we further design a compatible quantized synchronization and aggregation mechanism to efficiently support low-bit quantized tensors.

(2) **Collective Communication Scheduling Optimization:** We systematically analyze the All-Reduce communication process in cross-data-center scenarios, improve upon NCCL’s CollNet communication mechanism, and propose a multi-level pipeline communication scheduling strategy that fully exploits heterogeneous bandwidth resources between and within clusters, significantly accelerating the communication process.

(3) **Computation-Communication Overlap Optimization:** We quantitatively analyze the degree of computation-communication overlap in hybrid parallelism strategies, compare the overlap differences between cross-data-center communication and intra-data-center

communication, and improve the overlap degree by redesigning the gradient synchronization mechanism in hybrid parallelism mode.

Keywords: Distributed Training, Cloud-Edge-Device Collaboration, Cross-Domain Communication, Communication Optimization, Large Language Models

目 录

第一章 绪论	1
1.1 研究背景	1
1.1.1 大语言模型的规模演进与分布式训练需求	1
1.1.2 云边端协同训练的兴起	1
1.1.3 跨域通信的层次异构性	2
1.2 云边端协同分布式训练的通信挑战	2
1.3 研究问题与目标	3
1.4 本文主要工作与创新点	4
1.5 论文结构安排	5
第二章 国内外相关研究	6
2.1 分布式训练并行策略与系统框架	6
2.1.1 数据并行与参数/状态分片	6
2.1.2 模型并行与流水线并行	6
2.2 跨域与跨数据中心训练	6
2.3 梯度压缩与通信量优化	7
2.3.1 稀疏化与误差反馈	7
2.3.2 自适应压缩与系统协同	7
2.4 集合通信优化与层次化调度	7
2.5 通信重叠与同步尾部优化	8
2.6 云边端协同训练范式与研究空白	8
2.6.1 云边端协同训练问题	8
2.6.2 云边端场景下的跨域训练策略的选择	9
2.6.3 本文后续研究路线	10
2.6.4 本文通信优化系统的整体论述：从单点优化到端到端协同	10
第三章 分布式优化器低比特量化方法	12
3.1 引言	12
3.1.1 与云边端协同的关联	12
3.1.2 分布式训练中的通信瓶颈	12
3.1.3 现有梯度压缩方法的局限性	13

3.2 问题分析与研究思路	13
3.2.1 算法核心思想	13
3.2.2 数学表达	14
3.3 详细方案设计与实现	15
3.3.1 量化过程	15
3.3.2 位打包优化	16
3.3.3 反量化与聚合	16
3.3.4 集成到分布式训练框架	17
3.4 实验与验证	19
3.4.1 实验设置	19
3.4.2 实验结果与分析	21
3.5 本章小结	25
第四章 基于流水线的层次化梯度通信调度策略	26
4.1 引言	26
4.1.1 异构集群通信特征	26
4.1.2 传统同步算法的局限	26
4.2 问题分析与研究思路	27
4.2.1 层次化通信拓扑	27
4.2.2 流水线分块策略	28
4.2.3 双流并行机制	28
4.3 详细方案设计与实现	29
4.3.1 阶段一：预热阶段（Warmup）	29
4.3.2 阶段二：流水线主循环（Pipelining）	29
4.3.3 阶段三：冷却阶段（Cooling）	29
4.4 理论性能与开销分析	29
4.4.1 时间复杂度	29
4.4.2 空间复杂度	31
4.5 系统集成与工程优化	31
4.6 实验与验证	31
4.6.1 实验环境	31
4.6.2 通信性能对比	32

4.6.3 端到端训练加速	33
4.6.4 块大小敏感性分析	34
4.6.5 带宽不对称性影响	35
4.6.6 低质网络全面实验设计	36
4.7 技术优势与创新点	38
4.8 局限性与未来工作	38
4.9 本章小结	38
第五章 通信受限场景下的混合并行通信掩盖策略	39
5.1 引言	39
5.1.1 并行骨架的选择：跨 DC DP 优于跨 DC PP	40
5.1.2 同步尾部成为主要瓶颈	41
5.2 问题分析与研究思路	42
5.3 详细方案设计与实现	43
5.4 前缀触发的异步梯度同步	44
5.4.1 一次迭代只触发一次 All-Reduce	44
5.4.2 截断点 τ 的选择规则	44
5.5 预测误差修正 (Predictive Error Correction)	45
5.5.1 前缀梯度与完整梯度	45
5.5.2 发送缓冲与误差更新	45
5.6 工程实现与系统集成	46
5.6.1 与 1F1B 流水线的集成位置	46
5.6.2 通信 stream 与计算 stream 的隔离	47
5.6.3 误差缓冲的存储与开销	47
5.7 理论分析	47
5.8 实验与验证	48
5.8.1 实验设置	48
5.8.2 端到端吞吐	48
5.8.3 收敛曲线与消融	49
5.9 本章小结	51
第六章 协同通信优化系统：集成与最终实验测试	52
6.1 引言	52

6.2 问题分析与研究思路	52
6.2.1 统一符号与状态记号	53
6.3 详细方案设计与实现	53
6.4 系统集成接口与执行流程	54
6.4.1 模块接口与状态变量约定	54
6.4.2 运行时决策与回退机制	55
6.4.3 系统控制器主循环（伪代码）	55
6.4.4 工程实现细节与开销讨论	56
6.5 实验与验证	56
6.5.1 测试目标与判据	56
6.5.2 最终测试流程与测试矩阵	56
6.5.3 最终测试结果汇总	57
6.5.4 结果讨论：适用边界与失效模式	59
6.5.5 可复现性与部署建议	59
6.6 本章小结	59
参考文献	60
攻读硕士学位期间取得的成果	63
致谢	64
作者简介	65

插图清单

图 3.1 大规模分布式训练中的通信瓶颈	13
图 3.2 k-bit 随机舍入量化核心工作流程	14
图 3.3 位打包与解包过程示意图	16
图 3.4 k-bit 随机舍入量化在分布式框架中的集成架构	19
图 3.5 k-bit 与主流压缩方法的端到端性能对比	22
图 3.6 k-bit 在多模型端到端训练中的收益	23
图 3.7 FP32 与 4-bit 在代表性网络条件下的 validation loss 收敛曲线	24
图 3.8 k-bit 量化方法的收敛性与训练加速比验证（以 LLaMA-7B 为例）	25
图 4.1 异构带宽不对称性导致的通信瓶颈	26
图 4.2 层次化 All-Reduce 通信拓扑与数据流向	27
图 4.3 流水线层次化 All-Reduce 时序图（ar: All-Reduce, bc: Broadcast）	28
图 4.4 流水线调度器在训练流程中的集成架构	31
图 4.5 端到端训练性能对比与耗时分解	33
图 4.6 块大小对通信性能的影响	34
图 4.7 固定张量（1024 MB）下，不同 chunk 数在带宽不对称场景中的加速比	35
图 4.8 实验组 A：跨域带宽退化设置	36
图 4.9 实验组 B：跨域 RTT 分级设置	37
图 5.1 跨数据中心流水线并行：数据必须汇入单入口 DC，产生入口瓶颈	39
图 5.2 跨数据中心数据并行 + 数据中心内流水线并行：各 DC 处理本地数据，仅跨 WAN 同步梯度	40
图 5.3 DP+PP 配置下每步计算/通信开销分解：跨 DC 同步成为主要等待来源	41
图 5.4 DP+PP 下的执行时间线对比：POLAR-SGD 通过前缀触发的异步 All-Reduce 缩短同步尾部	43
图 5.5 训练 loss 随 step 变化：POLAR-SGD 与基线高度一致，优于 LSGD 的偏移 ...	49
图 5.6 训练 loss 随 wall-clock time 变化：POLAR-SGD 更快进入低 loss 区间	50
图 5.7 消融实验：去除梯度缩放或误差反馈会带来轻微收敛退化或稳定性风险	50
图 6.1 三个通信优化模块协同机制与运行时决策闭环	54
图 6.2 固定模型与 batch 条件下，本系统与基线系统的吞吐、迭代时间与通信占比对比	57

图 6.3 本系统与基线系统的 step 对齐与 wall-clock 对齐 loss 曲线	58
图 6.4 高带宽/低 RTT 退化测试下的自动回退与一致性验证	58

附表清单

表 3.1	云边端协同场景的分层链路参数	20
表 3.2	对比实验方法集合与实现口径	20
表 3.3	多网络条件实验分组	20
表 3.4	多模型端到端实验矩阵	20
表 4.1	用于云边端协同场景的分层链路仿真参数	32
表 4.2	不同 All-Reduce 方法的通信性能对比	32
表 4.3	端到端训练性能对比	34
表 5.1	POLAR-SGD 记号表	42
表 5.2	实验设置	48
表 5.3	Cross-DC 约束下端到端吞吐	48
表 6.1	第六章系统集成的统一符号与状态记号	53
表 6.2	三模块系统集成中的接口与状态变量约定	54
表 6.3	不同网络环境下本系统与现有训练系统的最终对比结果	57

第一章 绪论

1.1 研究背景

1.1.1 大语言模型的规模演进与分布式训练需求

以 Transformer 架构^[1] 为核心的预训练范式推动了大规模语言模型与基础模型 (Foundation Model) 的快速演进^[2]。从 BERT (3.4 亿参数) 的双向预训练^[3], 到 GPT-3 (1750 亿参数) 的少样本学习^[4], 再到 PaLM (5400 亿参数) 的 Pathways 架构^[5]、LLaMA 系列 (7B 至 70B 参数, 开源高效训练)^[6], 以及近期 DeepSeek-R1 (6710 亿参数混合专家架构) 的推理增强模型^[7], 语言模型的参数规模在短短数年间完成了数量级的跨越。Kaplan 等人通过实证揭示了模型性能与训练计算量、参数量之间的幂律扩展关系 (Scaling Law)^[8], 进一步奠定了“大规模训练范式”的理论基础; Hoffmann 等人则在此基础上给出了计算最优的参数量与训练数据量匹配准则 (Chinchilla 定律)^[9], 两项工作共同引导了产业界在算力投入不断增大背景下的“大模型训练策略”。基础模型的概念^[2] 进一步将这一范式推广至视觉、多模态、科学计算等领域, 极大地拓宽了大规模分布式训练的应用场景。

随着参数规模持续攀升, 单 GPU 乃至单节点训练已无力承载百亿至千亿参数模型。以 GPT-3 (175B) 为例, BF16 精度下仅存储模型参数即需约 350 GB, 结合 Adam 优化器的梯度与状态, 显存需求可超过 2 TB, 远超单张 GPU 的 80 GB 容量上限。大规模分布式并行训练因此成为必要的基础设施。学术界与工业界共同构建了多维并行体系: 数据并行 (Data Parallelism, DP) 通过批次切分实现规模扩展并以 All-Reduce 同步梯度; 张量并行 (Tensor Parallelism, TP) 在算子粒度切分权重矩阵以降低单设备显存压力^[10]; 流水线并行 (Pipeline Parallelism, PP) 将模型层划分为多个 stage 并以 micro-batch 串行注入实现层间并行^[11,12]; 专家并行 (Expert Parallelism, EP) 通过条件计算动态路由实现超大模型扩展^[13]; ZeRO 技术通过对优化器状态、梯度与参数进行分层分片, 显著降低每节点显存占用^[14]。Narayanan 等人在千亿参数模型上展示了 DP+TP+PP 组合并行 (3D Parallelism) 的规模化工程实践^[15], PyTorch 生态进一步提供了对分片并行的工程化支持^[16,17], 形成了覆盖“计算、通信、内存”三个维度的完整分布式训练工程体系。

1.1.2 云边端协同训练的兴起

训练数据与算力资源正从“集中式单数据中心”走向“云-边-端协同”的跨域形态, 驱动力来自多个层面。其一, 数据隐私与合规约束 (如 GDPR、HIPAA、金融数据本地

化规定)要求敏感数据就地留存于边缘站点或终端设备,无法集中回传至中心云;其二,业务地理分布(跨国服务、多区域监管合规)要求在多个数据中心并行维护训练与推理能力;其三,5G 边缘节点、企业私有云与端侧加速器的快速部署在全球范围形成了分散而异构的算力资源池;其四,出于弹性成本与算力利用率的考量,企业往往希望在训练高峰期借助边缘节点动态扩展算力,在低峰期释放以节省成本。

在这一背景下,云-边-端协同训练涉及三类参与主体:**云侧**(Cloud)具有强算力(大规模 GPU 集群)与大容量存储,承担主要的计算任务与全局模型状态维护;**边缘侧**(Edge)通常为微型数据中心或边缘服务器,可提供中等算力并具备就近处理本地数据的能力;**端侧**(Device)如手机、车载计算单元,算力有限但数据产生丰富,参与局部计算或梯度上传。三者构成“端→边→云”的逐级汇聚拓扑,在通信特性上呈现显著的层次异构性。

1.1.3 跨域通信的层次异构性

云-边-端协同训练在网络层形成明显的多层次带宽与时延差异。域内链路方面,同一节点内的 NVLink 互联带宽可达 600 GB/s,节点间高性能以太网或 InfiniBand HDR RDMA 可提供 200 Gb/s 以上的低时延互联;而跨域链路方面,典型的边缘↔云或边缘↔边缘的 WAN 带宽通常仅有 1 - 100 Gb/s,RTT 在数十至数百毫秒量级,且受网络拥塞与路由波动影响,抖动明显。这种“域内高速—域间受限”的分层互联特征,使得传统上为同质高速网络设计的扁平化集合通信策略(如 Ring All-Reduce)难以有效适配,梯度同步极易进入迭代关键路径,形成显著的通信瓶颈^[18]。

1.2 云边端协同分布式训练的通信挑战

在同步数据并行训练框架中,每次迭代末尾需通过 All-Reduce (或其等价变体)聚合全局梯度均值。当训练跨域展开时,同步通信面临以下四个系统性挑战:

挑战一: 跨域带宽受限与梯度规模的耦合放大。大模型梯度同步的通信量与参数规模线性正相关:以 BF16 精度计,175B 参数模型每步梯度约需 350 GB 的单向传输量。在采用层次化 All-Reduce 时,跨域部分至少需传输一次完整的归约梯度(Reduce-Scatter 阶段后)与广播结果(All-Gather 阶段),总量仍达数百 GB 量级。若可用跨域带宽仅有 10 - 30 Gb/s,即使通过梯度量化将通信量压缩至原始的 1/4 至 1/32,跨域传输时间仍可能占据总迭代时间的相当比例,出现“算力翻倍但吞吐停滞”乃至“加节点反而更慢”的现象。

挑战二：高 RTT 与链路抖动加剧同步尾部延迟。WAN 链路的高 RTT（跨大洲可达 50 - 200 ms）与抖动（10 - 50 ms 量级）会将集合通信的启动开销与同步等待放大至不可忽略的程度。在同步 DP 框架中，每步训练须等待最慢节点完成通信（“木桶效应”），而链路抖动带来的随机慢节点（straggler）效应会使吞吐分布出现长尾，系统整体有效吞吐受最慢单次通信主导。在 DP+PP 混合并行场景中，流水线末尾阶段的 All-Reduce 本已处于计算关键路径之外，高 RTT 进一步扩大尾部阻塞窗口，加重设备空闲。

挑战三：分层拓扑与扁平化集合通信不匹配。传统扁平化 Ring/Tree All-Reduce 将各节点视为对等参与者，无法感知“域内高速—域间低速”的拓扑差异，常导致域内高速链路的聚合潜力被域间低速瓶颈拖累。层次化 All-Reduce（域内 Reduce-Scatter → 跨域 All-Reduce → 域内 All-Gather）能够将跨域通信量从正比于全局节点数降为仅传输一份归约结果，显著降低跨域流量；但如何进一步对层次化各阶段进行流水化与并发调度，以充分利用域内高速带宽并最大化跨域传输的重叠度，仍是调度层面的核心问题。

挑战四：混合并行下计算-通信重叠窗口受限。在数据并行框架中，常见的优化手段是将梯度按层分组为若干 bucket，在反向传播过程中逐批启动 All-Reduce，使通信与后续层的反向计算形成时间重叠。然而在跨域高 RTT 环境下，最后若干 bucket 对应的反向层计算已完成，而其 All-Reduce 尚未结束，形成不可压缩的同步阻塞间隙（synchronization gap）。在 DP+PP 混合并行的流水线 flush 阶段，此问题尤为突出：PP 进入排空阶段后已无新 micro-batch 可投入计算，跨域 All-Reduce 仍在进行，所有设备不得不空等，直接拉低整体硬件利用率。

上述四个挑战共同表明：面向云边端协同的跨域分布式训练，需要在通信量、集合通信调度以及计算-通信重叠三个层面协同优化，方能在不牺牲训练语义的前提下有效提升端到端效率。

1.3 研究问题与目标

本文面向云边端协同的跨域分布式训练场景，围绕“如何在跨域低带宽、高 RTT、强抖动的约束下，降低梯度同步对关键路径的侵入”这一核心问题，聚焦以下三个子目标：

目标一（通信量压缩）：在不牺牲模型收敛稳定性的前提下，通过低比特量化与误差补偿机制，最大化压缩跨域链路上的梯度有效传输字节数，并设计可灵活配置的精度-压缩率折中方案，以适应不同场景下的带宽约束与收敛要求。

目标二（分层调度）：利用域内/域间带宽不对称性，构建与分层拓扑相匹配的集合通信调度机制。通过张量分块（**chunking**）与双流并行（**dual-stream**）将“域内归约—域间同步—域内广播”的各阶段流水化并发执行，以提升跨域同步的并行度并降低整体延迟。

目标三（重叠优化）：在混合并行与高 RTT 条件下，通过重新设计梯度同步的触发时机，将 All-Reduce 从迭代末尾的不可重叠窗口前移至更早的计算阶段，配合预测误差修正在保持“每迭代一次全局同步”语义的前提下，显著减少迭代末端的同步阻塞，改善整体 **time-to-convergence**。

为便于刻画云边端协同的跨域网络特征并开展可控实验，本文以跨数据中心训练作为典型强约束实例：跨 DC 场景的 WAN 带宽与 RTT 特征与边缘↔云或边缘↔边缘的典型互联相吻合，且可在实验中通过流量整形工具（**tc-netem**）精确复现，便于系统性评估各优化手段的收益。

1.4 本文主要工作与创新点

本文从“通信量—调度—重叠”三个层次对跨域分布式训练通信优化进行系统性研究，主要工作与创新点如下：

工作一（第 3 章）：面向跨域带宽瓶颈的低比特量化通信机制。针对跨域链路传输大模型梯度的高开销问题，本文提出基于 **k-bit** 随机舍入量化的分布式优化器，支持可配置位宽 $b \in \{1, 2, 4, 8, 16\}$ 。在此基础上，设计配套的位打包（**bit-packing**）、量化梯度聚合与误差补偿（**error compensation**）机制，实现对量化张量的高效通信支持。在受限链路上，16-bit 至 1-bit 的量化最高可将通信数据量降至原始的 1/32，同时通过误差反馈维持收敛稳定性，在压缩率与优化质量之间提供可调折中。

工作二（第 4 章）：张量分块与双流并行驱动的层次化流水线 All-Reduce。针对“域内快、域间慢”的分层互联特征，本文提出将层次化 All-Reduce 的“域内 Reduce-Scatter—域间 All-Reduce—域内 All-Gather”三个阶段分解为可流水化执行的分块调度策略，并通过双 CUDA 流并行驱动域内与域间通信的并发执行，充分利用域内高速带宽对跨域同步延迟的掩盖窗口，显著提升跨域同步的重叠度与整体通信效率。

工作三（第 5 章）：前缀触发异步 All-Reduce 与预测误差修正机制。针对高 RTT 环境下混合并行训练末尾的同步尾部问题，本文提出将 All-Reduce 的触发时机从反向传播完成后前移至前向传播中某一前缀层完成后立即启动，利用前向与后续反向传播之间的计算窗口覆盖跨域通信延迟；对尚未完成反向传播的参数引入预测误差修正，保证全局

同步的统计等价性。该机制在“每迭代一次全局同步”的宏观语义下，将跨域 All-Reduce 从迭代关键路径末端剥离，有效改善端到端吞吐与收敛速度。

1.5 论文结构安排

全文组织如下：

- **第 1 章：**介绍研究背景、问题挑战与本文贡献。
- **第 2 章：**综述云边端协同训练、分布式并行与系统框架、梯度压缩、集合通信调度、通信重叠与尾部优化等方向的代表性工作，并基于此展开分析，确定研究问题、研究路线。
- **第 3 章：**给出低比特量化分布式优化器与低比特位的通信机制的设计与实现，并讨论其在跨域链路上的适用性。
- **第 4 章：**提出并分析层次化流水线 All-Reduce 调度策略，给出工程集成与实验评估。
- **第 5 章：**提出通信受限场景下的混合并行通信掩盖策略，并在跨域约束下给出实验验证与分析。
- **第 6 章：**将低比特量化、层次化流水线通信与计算-通信掩盖整合为统一系统，并给出系统级最终实验与测试汇总。

第二章 国内外相关研究

本章围绕“云边端协同的跨域分布式训练通信优化”主题，综述相关研究脉络。为便于对齐本文的三项工作，本章按“并行训练与系统框架—跨域/跨数据中心训练—通信量压缩—集合通信调度—通信重叠与尾部优化—云边端协同训练范式”的逻辑组织。

2.1 分布式训练并行策略与系统框架

2.1.1 数据并行与参数/状态分片

数据并行（DP）通过复制模型并切分数据批次实现扩展性，但其关键成本来自每步梯度同步。为降低显存占用并支撑更大模型，研究与工业界提出了多种参数/优化器状态分片策略，例如 ZeRO 系列工作通过对优化器状态、梯度与参数进行分层分片，实现更高的内存效率^[14]。PyTorch Distributed 与 Fully Sharded Data Parallel（FSDP）提供了工程化的分片并行支持与规模化经验总结^[16,17]。

从云边端协同角度看，分片与内存优化使得边缘节点也能承载更大模型或更大 batch，从而为“边缘侧参与训练与聚合”提供系统基础；但在跨域链路受限时，DP 的同步通信仍可能成为端到端瓶颈，需进一步引入通信优化。

2.1.2 模型并行与流水线并行

当模型规模超过单设备显存时，常采用张量并行、流水线并行等方式分解计算。Megatron-LM 展示了在大规模 GPU 集群上训练数十亿参数语言模型的系统实践^[10]；GShard 等工作探索了更大规模模型的自动切分与条件计算^[13]。

流水线并行通过将网络层划分为多个 stage，并以 micro-batch 的方式串行注入、并行执行。GPipe 与 PipeDream 是两类代表性流水线并行系统：前者强调高效的流水线执行与可扩展性，后者提出更一般化的流水线并行与调度策略^[11,12]。近年来还有以减少流水线气泡为目标的改进工作，例如 Zero Bubble Pipeline Parallelism^[19]。

在云边端协同场景中，流水线并行通常更适合在同一域内（如边缘域内或云内）使用，以避免跨域传输激活带来的高频通信；而跨域层面更常采用 DP 语义，只在迭代末进行一次梯度同步，从而将跨域通信频率降到最低。

2.2 跨域与跨数据中心训练

跨数据中心/地理分布式训练可视为云边端协同中的“域间互联”的典型形态：带宽相对受限、RTT 与抖动显著，且网络与计算资源分散。围绕跨域训练的系统问题，近年

来出现了面向跨数据中心流水线调度与训练框架的研究，例如 CrossPipe 针对跨数据中心训练提出更优的流水线调度策略^[20]；GeoPipe 探索了地理分布式 LLM 训练框架中的流水线并行增强^[21]；JEEVES 从调度视角研究跨 DC 训练的系统协调问题^[22]。

除深度学习训练外，宽域数据分析任务也积累了大量关于跨域数据并行作业优化的经验，例如 PIXIDA 针对宽域数据分析任务的优化机制为理解 WAN 约束下的并行作业提供了参考^[23]。

2.3 梯度压缩与通信量优化

梯度压缩通过减少每步需要传输的数据量来降低通信时间，主要路线包括稀疏化与量化两类。

2.3.1 稀疏化与误差反馈

稀疏化方法通过只传输 Top- k 或随机采样的一部分梯度元素来减少通信量，但会引入偏差或方差增大。Sparsified SGD with Memory 提出了带记忆的稀疏化机制，通过误差反馈（error feedback）累积未发送信息以改善收敛^[24]。后续研究进一步从理论与实践上分析了误差反馈在压缩 SGD 中的关键作用，例如 Error Feedback Fixes SignSGD 等工作说明误差反馈能够修正符号压缩等方案的偏差，使其恢复稳定性^[25]。

在云边端协同场景中，误差反馈对跨域受限链路尤为重要：当端↔边、边↔云链路带宽波动时，误差反馈可作为一种“残差信道”，将被压缩或延迟的梯度信息在后续迭代中逐步补偿，从而在系统效率与优化稳定性之间提供可控折中。

2.3.2 自适应压缩与系统协同

为了在不同网络与梯度分布下稳定获益，研究还探索了更自适应的稀疏压缩与系统协同设计。例如 PacTrain 将剪枝与自适应稀疏梯度压缩结合，用于更高效的集合通信^[26]。误差反馈也被扩展到更一般的预条件器与优化器状态压缩中^[27]。

本文第 3 章聚焦低比特量化路线：在尽量保持训练语义的前提下，将跨域同步通信的字节规模压缩到更低水平，并在后续章节与“分层调度”和“重叠”机制组合，以适配云边端跨域约束。

2.4 集合通信优化与层次化调度

All-Reduce 是 DP 中最关键的集合通信原语之一，其实现方式（如 ring、tree 以及分层变体）决定了通信带宽利用率与同步尾部。对于域内快、域间慢的分层拓扑，层次化

All-Reduce 通过“域内聚合—域间同步—域内分发”减少跨域通信量，并将跨域同步集中在少量代表进程之间，从而更好地匹配带宽不对称性。

跨域训练研究进一步表明，仅有层次化还不够：当域间 RTT 高且通信启动开销显著时，需要通过流水线化、分块、双流并发等方式提高并行度并隐藏启动开销^[20]。本文第 4 章从工程可实现的角度给出分块与双流层次化流水线调度策略，并在实验设置中以“域内/域间链路”参数来对齐云边端协同拓扑。

2.5 通信重叠与同步尾部优化

在常规数据中心环境中，框架常通过 bucket 化在反向传播中逐步启动 All-Reduce 以实现计算-通信重叠；但在 WAN 环境下，高 RTT 可能使得最后几个 bucket 无法被剩余计算掩盖，形成同步尾部。围绕“低通信频率”与“重叠通信”的设计，近年来出现了多类跨域训练方法与系统，例如 OpenDiLoCo 以低通信频率为目标探索全球分布式训练框架^[28]，而 Streaming DiLoCo 进一步强调通信与计算的重叠以提升系统效率^[29]。

除低通信频率路线外，另一条关键思路是改变同步触发时机：将一次关键的同步从迭代末尾迁移到更早的可重叠窗口中，从而缩短尾部。本文第 5 章采用“前缀触发的异步 All-Reduce + 预测误差修正”的机制，目标正是将跨域同步从关键路径尾部剥离，并在保持“每迭代一次同步”的宏观语义下获得更稳定的系统收益。

2.6 云边端协同训练范式与研究空白

2.6.1 云边端协同训练问题

结合本文的背景，可以看到，云边端协同训练是一种跨域资源联合训练：

这种跨域性首先体现在网络层：域内链路通常高带宽、低时延，而域间链路普遍低带宽、高 RTT 且抖动更大；其次体现在数据层：训练样本天然分布在云侧数据湖、边缘站点与端侧来源，不宜也不总能集中回传；再次体现在算力层：参与训练的设备来自不同自治域，调度能力和负载上限差异显著。

因此，云边端协同训练的主要矛盾不是“如何在同构高速网络里扩规模”，而是“如何在异构域间链路约束下组织同步与并行”。这一定义也给出本章的阶段性的结论：需要先回答“跨域并行策略如何选择”，再讨论“在既定策略下如何做通信优化”。

2.6.2 云边端场景下的跨域训练策略的选择

综合并行训练与跨域训练研究, 可将策略选择归结为两个判断维度: 跨域通信对象是否随 batch 增长, 以及该通信是否持续位于关键路径。

当采用跨域流水线并行 (Cross-domain PP) 时, 模型 stage 会跨域切分, 域间通信对象主要是激活及其反向梯度; 这一方案在参数规模受限场景中有助于继续扩展模型, 但跨域通信与样本流强耦合, WAN 抖动容易沿流水线级联放大。相比之下, 跨域数据并行 (Cross-domain DP) 将前反向计算留在各域本地, 域间交互集中为梯度或参数同步, 对网络抖动更鲁棒, 但前提是同步路径具有足够高效的通信实现。

本文通过量化推导给出更具体的比较关系。设全局 batch 为 B , 参数规模为 P , 每参数通信字节为 α , 跨域切分时每样本跨域激活字节为 A :

跨域 DP 的通信量近似为 $V_{DP} \approx \alpha P$, 在模型固定时对 B 近似不敏感; 而跨域 PP 的通信量近似为 $V_{PP} \approx \Theta(B * A)$, 会随 B 线性增长。

据此可得到如下选择原则:

- 当目标是继续突破单域模型容量上限, 且域间链路相对稳定时, 可考虑跨域 PP;
- 当目标是在受限 WAN 条件下保持稳定吞吐与训练语义可控时, 应优先跨域 DP。

在大 batch 训练下, 跨域 PP 的 WAN 压力会持续放大; 而跨域 DP 的固定同步开销更容易被计算摊薄。因此, 许多系统实践逐步收敛到“跨域 DP + 域内 PP”这一分层组织方式: 跨域层面优先保证稳定吞吐, 域内层面再用流水线并行扩展模型容量与设备利用率。

在此基础上, 本文进一步说明为何采用“跨域数据并行”而非“跨域流水线并行”。本文的问题设定是“云边端协同下的大规模数据训练”, 核心目标是在真实跨域链路约束下提升端到端吞吐并保持优化语义稳定。本文选择跨域 DP 的原因主要有三点:

- 数据局部性: 各域可利用本地数据独立完成前反向计算, 避免样本与高频激活长期占用 WAN;
- 关键路径可控: 域间交互收敛为同步阶段, 便于对关键路径进行可测量、可治理的优化;
- 与后续方法匹配: 第 3 章、第 4 章、第 5 章分别优化“同步数据量、同步调度效率、同步尾部重叠”, 三者围绕同一域间同步对象形成协同闭环。

本文并不否定流水线并行本身，而是将其定位为 **域内扩展手段**：在单域内利用高速互联做 PP/TP，在跨域层面采用 DP 语义，以同时兼顾模型规模扩展、网络鲁棒性与工程可落地性。

2.6.3 本文后续研究路线

在以 DP 作为基础架构下的云边端协同训练既可以采用集中式跨域同步（强调全局一致性），也可以采用更弱同步或更强异步的范式。例如 Local SGD 通过降低通信频率以减少开销，并在部分设置下获得较好的通信-收敛折中^[30]；联邦学习（Federated Learning）强调数据不出端，并在非独立同分布与系统异构下发展出多种控制偏差与加速收敛的算法（如 SCAFFOLD）^[31]。

然而，在“云-边-端协同的大模型训练”这一目标下，系统通常希望在可控的语义约束下保持较强的一致性（例如每步一次跨域同步或周期性同步），以避免优化轨迹偏离带来的不可控风险；同时还希望尽可能复用主流深度学习框架与通信后端，降低工程集成成本。因此，仍需要面向分层拓扑与跨域链路的通信优化：

- 在跨域同步不可避免时，如何进一步降低通信体积并控制误差累积？
- 在域内/域间带宽高度不对称时，如何设计集合通信调度，优化集合通信执行时间？
- 在高 RTT 场景下，如何将同步从尾部迁移到可重叠窗口，并用误差修正保持稳定语义？

2.6.4 本文通信优化系统的整体论述：从单点优化到端到端协同

为回答上述问题，本文并非提出彼此割裂的三个“技巧”，而是构建了一个面向云边端跨域协同训练的 **通信优化系统**。该系统以“跨域 DP（域间）+ 多维混合并行（域内 DP/TP/PP）”为执行底座，在统一训练语义下把通信优化拆解为三个可组合组件，并通过接口与时序协同形成闭环。

组件一：低比特量化通信（第 3 章）在不改变训练主流程的前提下，对跨域同步张量进行低比特表示与误差补偿，将跨域传输字节数显著压缩。其系统角色是“先降体积”，直接降低 WAN 有效负载与拥塞概率，为后续调度与重叠创造更大的时间窗口。

组件二：分层集合通信与流水化调度（第 4 章）针对“域内快、域间慢”的网络事实，设计域内/域间分层 All-Reduce，并在域间阶段采用分块、流水、并发传输策略，提升慢链路利用率并降低启动开销占比。其系统角色是“再提效率”，把已经压缩后的通信数据以更匹配拓扑的方式传输，缩短同步主干时长。

组件三：DP+PP 混合并行下的计算-通信掩盖（第 5 章）结合域内大模型多维混合并行配置，重构跨域同步触发时机与执行路径，将部分同步从迭代尾部前移到可重叠区

间，并通过预测/误差修正维持优化稳定。其系统角色是“最后去尾部”，把剩余通信进一步掩盖到计算过程中，减少 GPU 空等。

从系统视角看，三者满足明确的协同顺序与乘性收益关系：

- 量化先将通信量做“减法”；
- 分层流水调度对剩余通信做“结构化加速”；
- 重叠机制对残余尾部做“关键路径剥离”。

因此，本文解决的不是单一通信算子加速问题，而是云边端跨域协同训练中的端到端瓶颈治理问题：在保持跨域 DP 语义一致性与工程可部署性的前提下，系统性缓解“带宽受限、时延偏高、抖动显著”带来的训练吞吐下降与尾部阻塞。

综上，本文三项工作在统一框架下协同运作：第 3 章降低跨域通信体积，第 4 章提升分层调度与带宽利用率，第 5 章降低同步尾部并提升重叠程度，最终形成一套可落地、可组合、可扩展的云边端跨域训练通信优化系统。

第三章 分布式优化器低比特量化方法

3.1 引言

在分布式深度学习训练中，梯度通信开销已成为制约训练效率的关键瓶颈。传统的 AllReduce 操作需要在各个计算节点间传输完整的 32 位浮点梯度，在大规模模型和多节点环境下，通信时间往往占据训练过程的主要部分。为降低通信开销，本文引入了 **k-bit 随机舍入量化方法**，将梯度压缩至可配置的低比特表示 ($b \in \{1, 2, 4, 8, 16\}$)，可在压缩率与收敛稳定性之间进行灵活折中。

3.1.1 与云边端协同的关联

云边端协同训练的一个典型形态是：端侧设备持续产生数据，边缘侧承担就近训练/聚合或作为训练入口，云侧提供大规模算力与全局同步能力。在该场景中，训练通信呈现明显的“分层链路”特征：边缘/云内链路可使用较高带宽的局域互联，而端到边、边到云往往受限于接入网与广域网，表现为更低带宽、更高 RTT 以及更大的抖动。

在这种分层拓扑下，梯度同步的瓶颈常由跨域链路主导，尤其当端侧或边缘侧参与数据并行 (DP) 时，同步梯度需要跨“端→边”或“边→云”链路传输。低比特量化的价值因此不仅体现在跨数据中心场景，也天然适用于云边端协同：它以最小的语义改动降低跨域链路上的通信体积，从而直接缓解接入/广域链路的带宽瓶颈，并降低通信排队与尾部等待对端到端时延的放大效应。

3.1.2 分布式训练中的通信瓶颈

随着深度学习模型规模的不断增长，单机训练已无法满足大规模模型的需求。以 GPT-3 为例，其参数量达到 1750 亿，模型大小超过 350GB，单次前向传播就需要数百 GB 的显存。这使得分布式训练成为必然选择。然而，在分布式训练过程中，梯度同步通信成为了新的性能瓶颈。

在标准的数据并行训练模式下，每个 worker 节点独立完成前向和反向传播，然后通过 AllReduce 操作同步梯度。对于包含 N 个参数的模型，每次迭代需要传输的数据量为：

$$\text{Communication Volume} = N \times \text{sizeof(float32)} = N \times 4 \text{ bytes} \quad (3.1)$$

以 ResNet-50 (约 2560 万参数) 为例，每次迭代需要传输约 97.7 MiB 数据。而对于 GPT-3 规模的模型，单次通信量约为 651.9 GiB (约 667,572 MiB)，在带宽受限的跨数据中心场景下，通信时间可能占据总训练时间的 80% 以上。

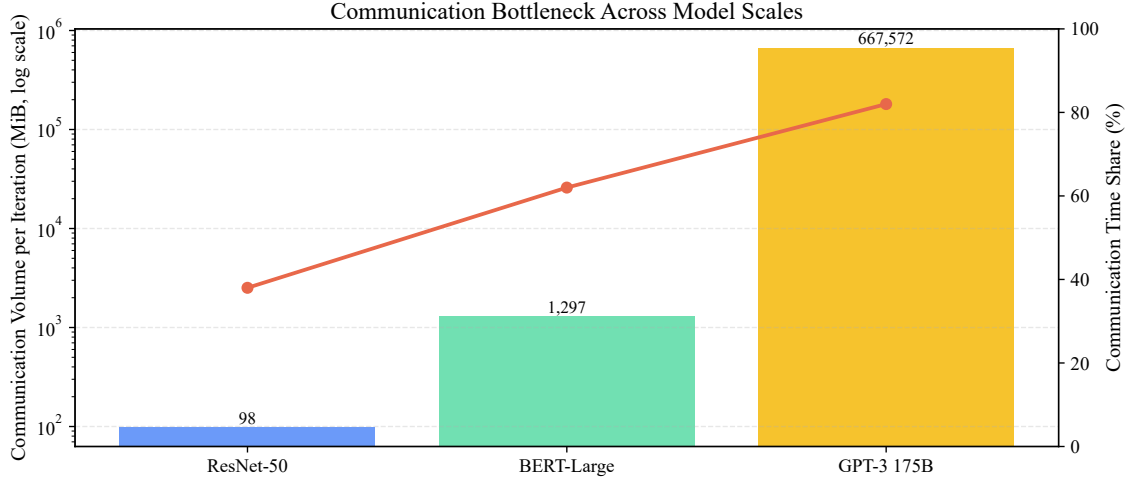


图 3.1 大规模分布式训练中的通信瓶颈

3.1.3 现有梯度压缩方法的局限性

为降低通信开销，学术界和工业界已经提出了多类梯度压缩方法，但在跨域受限链路场景中仍存在明显短板。以梯度稀疏化为例，尽管 Top-k 或阈值筛选能够在数值上显著压缩传输体积，但额外的稀疏索引编码与传输会引入新的开销，并在低带宽、高 RTT 环境中放大同步尾部，导致收敛效率下降。低精度量化路径则具有更好的工程可用性，FP16 在工业训练中已较成熟，但压缩收益通常只有 2 倍；INT8 虽可进一步提升到约 4 倍，却对量化尺度、裁剪区间和反量化策略更敏感，稍有不妥就可能损伤收敛稳定性。固定阈值量化方法在实现上最为直接，但由于缺乏对层间梯度尺度差异和训练阶段动态变化的自适应机制，往往难以同时满足高压缩率与稳定优化这两个目标。

相比之下，本文采用的 k-bit 量化方法结合了 Adam 风格的动量归一化与随机舍入策略，能够在保证收敛性的同时支持多档压缩精度：1-bit 强压缩、2/4-bit 均衡压缩、8/16-bit 低扰动压缩。

3.2 问题分析与研究思路

3.2.1 算法核心思想

k-bit 量化方法借鉴了 Adam 优化器的动量机制，通过维护梯度的一阶矩估计 m 和二阶矩估计 v ，将连续梯度映射到 2^b 个离散量化级别。该方法并非简单的线性截断，而是先利用指数移动平均保留历史梯度统计，再通过 m 与 v 的比值进行自适应归一化，使不同层、不同阶段的梯度都能被映射到统一数值区间；在此基础上，采用相邻量化级间

的随机舍入来维持无偏性，并将本轮量化残差注入后续迭代，形成可持续的误差补偿通道。由此，算法在“压缩强度”与“优化稳定性”之间建立了更稳健的平衡关系。

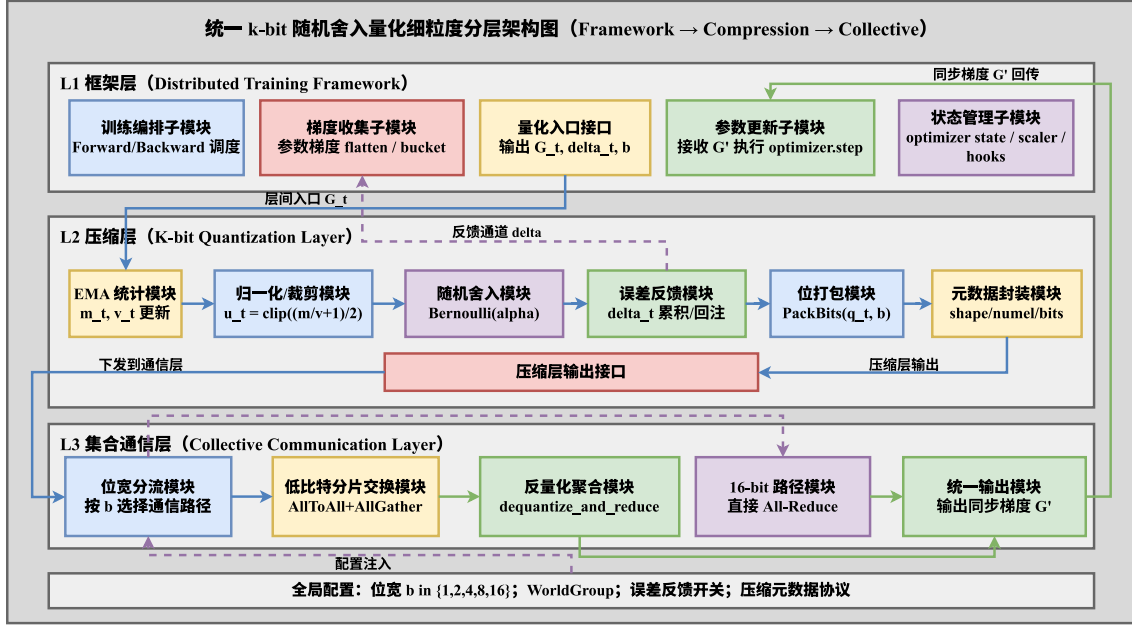


图 3.2 k-bit 随机舍入量化核心工作流程

3.2.2 数学表达

设第 t 次迭代的梯度为 g_t ，量化比特宽度为 b ，量化过程可表示为：

$$\begin{aligned}
 m_t &= \beta \cdot m_{t-1} + (1 - \beta) \cdot g_t \\
 v_t &= \beta \cdot v_{t-1} + (1 - \beta) \cdot |g_t| \\
 u_t &= \text{clip}\left(\frac{1}{2} \left(\frac{m_t}{v_t + \varepsilon} + 1 \right), 0, 1\right) \\
 s &= 2^b - 1 \\
 y_t &= s \times u_t \\
 l_t &= \lfloor y_t \rfloor, \quad \alpha_t = y_t - l_t \\
 z_t &\sim \text{Bernoulli}(\alpha_t), \quad q_t = l_t + z_t
 \end{aligned} \tag{3.2}$$

其中， β 表示动量系数（默认取 0.999）， ε 表示数值稳定项（默认取 10^{-8} ）， $b \in \{1, 2, 4, 8, 16\}$ 表示量化位宽，而 $q_t \in \{0, 1, \dots, 2^b - 1\}$ 则对应离散化后的量化索引。

随机舍入后的归一化估计值为 $\hat{u}_t = \frac{q_t}{s}$ ，其对应的对称区间表示为 $\hat{g}_t = 2\hat{u}_t - 1$ 。由于 $E[z_t] = \alpha_t$ ，可得 $E[\hat{u}_t] = u_t$ ，从而保证量化无偏性。

算法 3.1: k-bit 随机舍入梯度量化算法

输入: 梯度张量列表 $G = \{g_1, g_2, \dots, g_n\}$, 位宽 b , 动量系数 β , 误差补偿标志 comp_flag

输出: 压缩张量列表 $C = \{c_1, c_2, \dots, c_n\}$

```

1: for 每个梯度张量  $g_i \in G$  do
2:   if 动量状态未初始化 then
3:      $m_i \leftarrow g_i \times (1 - \beta)$ 
4:      $v_i \leftarrow |g_i| \times (1 - \beta)$ 
5:   else
6:      $m_i \leftarrow \beta \times m_i + (1 - \beta) \times g_i$ 
7:      $v_i \leftarrow \beta \times v_i + (1 - \beta) \times |g_i|$ 
8:   end if
9:    $u_i \leftarrow \text{clip}((m_i/(v_i + \epsilon) + 1)/2, 0, 1)$ 
10:  if  $\text{comp\_flag} = \text{True}$  then
11:     $u_i \leftarrow \text{clip}(u_i + \delta_i, 0, 1)$ 
12:  end if
13:   $s \leftarrow 2^b - 1; y_i \leftarrow s \times u_i; l_i \leftarrow \lfloor y_i \rfloor$ 
14:   $\alpha_i \leftarrow y_i - l_i; z_i \sim \text{Bernoulli}(\alpha_i)$ 
15:   $q_i \leftarrow l_i + z_i$ 
16:   $\delta_i \leftarrow \delta_i + (u_i - q_i/s)$ 
17:   $c_i \leftarrow \text{PackBits}(q_i, b)$ 
18: end for
19: return  $C$ 

```

3.3 详细方案设计与实现

3.3.1 量化过程

梯度量化算法的核心流程如算法 3.1 所示。该算法接收待量化的梯度张量列表作为输入，输出压缩后的 k-bit 张量列表。

从实现逻辑看，算法首先在第 2-8 行完成一阶矩与二阶矩的动量维护，以指数移动平均方式持续吸收历史梯度统计；随后在第 9 行通过二者比值进行自适应归一化，将不同尺度的梯度映射到统一的 $[0, 1]$ 区间。若启用补偿机制，第 10-12 行会把历史量化残差 δ 回注到当前归一化值并进行截断约束，从而抑制误差累积。第 13-15 行执行基于伯努利采样的随机舍入，得到离散索引 $q_i \in \{0, 1, \dots, 2^b - 1\}$ ，该过程保障了量化的统计无偏性；最后在第 17 行按位宽进行紧凑打包，减少通信与存储开销。

3.3.2 位打包优化

为进一步减少内存占用，本实现采用通用位打包技术，将量化索引按比特流连续存储。对于位宽 b ，每个量化值占用 b bit，原始 float32 梯度可获得理论压缩比 $\frac{32}{b}$ 。

具体实现时，系统首先执行填充对齐：当总 bit 数不是 8 的整数倍时，在末尾补零到完整字节边界，以保证后续字节级读写的正确性。随后构造位权向量 w ，用于将 bit 序列映射到字节位置。

$$w = [2^7, 2^6, 2^5, 2^4, 2^3, 2^2, 2^1, 2^0] = [128, 64, 32, 16, 8, 4, 2, 1] \quad (3.3)$$

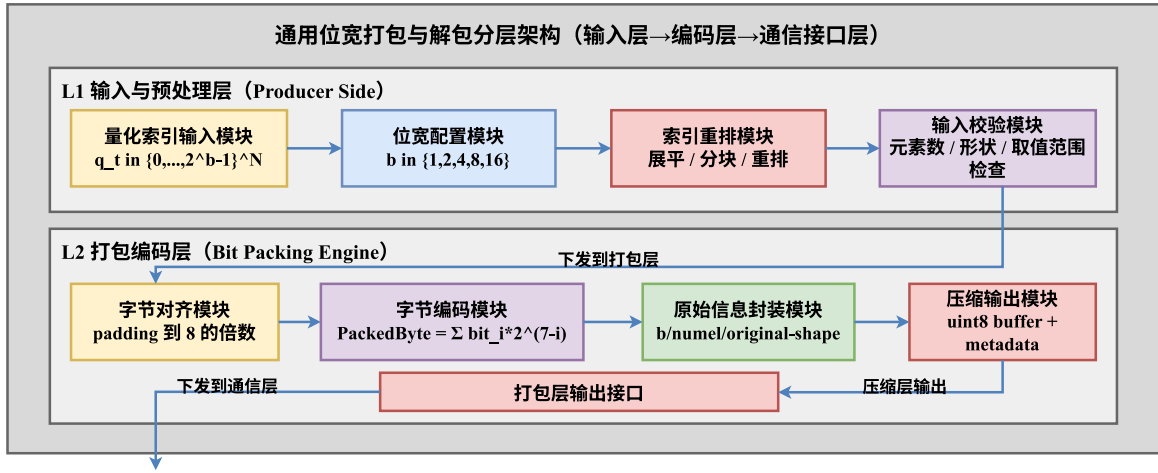


图 3.3 位打包与解包过程示意图

在此基础上，量化索引会被展开为 bit 流并按字节分组，与位掩码逐元素运算后得到 uint8 表示：

$$\text{PackedByte} = \sum_{i=0}^7 \text{bit}_i \times 2^{7-i}, \quad \text{bit}_i \in \{0, 1\} \quad (3.4)$$

该方法适用于不同位宽。若忽略对齐与元数据开销，通信量相对 float32 的理论压缩比分别为：1-bit 为 32 ×，2-bit 为 16 ×，4-bit 为 8 ×，8-bit 为 4 ×，16-bit 为 2 ×。

因此，该方法可在强压缩（1/2-bit）与高保真（8/16-bit）之间灵活切换，满足不同网络条件下的训练需求。

3.3.3 反量化与聚合

在接收端，各节点需要将来自所有 worker 的压缩梯度解包、聚合并恢复到可用于优化器更新的梯度空间。首先，压缩字节会被恢复为离散索引张量 $q \in \{0, 1, \dots, 2^b - 1\}$ ；实

算法 3.2: 集成量化的分布式训练循环

```

1: for 每个训练批次 do
2:   前向传播:  $y = f(x; \theta)$ 
3:   计算损失:  $\mathcal{L} = \text{Loss}(y, y_{\text{true}})$ 
4:   反向传播:  $\nabla \leftarrow \nabla_{\theta} \mathcal{L}$ 
5:   if 启用量化压缩 then
6:      $\nabla \leftarrow \text{QuantizedAdamReduce}(\nabla, b)$ 
7:   else
8:      $\nabla \leftarrow \text{AllReduce}(\nabla)$ 
9:   end if
10:  参数更新:  $\theta \leftarrow \theta - \eta \times \nabla$ 
11: end for

```

现上通过位运算完成高效解包，即将每个 uint8 字节展开为 8 位矩阵，再与位掩码 $w = [128, 64, 32, 16, 8, 4, 2, 1]$ 做按位与，并按位宽 b 重组为量化索引序列。该过程可写为：

$$q = \text{UnpackBits}(\text{PackedBytes}, b), \quad q \in \{0, 1, \dots, 2^b - 1\}^N \quad (3.5)$$

完成解包后，系统收集来自所有 W 个 worker 的量化索引并执行逐元素累加：

$$Q_{\text{sum}} = \sum_{w=1}^W q^{(w)} \quad (3.6)$$

其中 $q^{(w)} \in \{0, 1, \dots, 2^b - 1\}^N$ 表示第 w 个节点的量化索引， Q_{sum} 则对应参数维度上的离散级别和。随后进入反归一化阶段，将聚合后的离散值映射回原始梯度空间。设 $s = 2^b - 1$ ，其逆映射可写为：

$$g_{\text{recovered}} = 2 \times ((Q_{\text{sum}}/W)/s) - 1 \quad (3.7)$$

该公式将离散索引均值线性映射回 $[-1, 1]$ 区间，从而恢复梯度的符号与相对尺度信息。最终得到的 $g_{\text{recovered}}$ 可作为梯度的无偏估计参与参数更新。

3.3.4 集成到分布式训练框架

k-bit 量化方法已无缝集成到分布式训练流程中，替代了传统的 AllReduce 通信模式。整体架构设计遵循模块化原则，主要包含以下组件：

训练循环集成

在模型的前向传播和反向传播完成后，梯度同步阶段根据配置选择是否启用量化压缩，如算法 3.2 所示：

算法 3.3: QuantizedAdamReduce 通信流程

输入: 梯度张量列表 G , 位宽 b , 全局进程组WorldGroup

输出: 聚合后的梯度张量列表 G'

```

1: if  $b < 16$  then
2:   初始化量化器:  $Q \leftarrow \text{KBitAdamQuantizer}(b)$ 
3:   量化梯度:  $C \leftarrow Q.\text{quantize}(G)$ 
4:   for 每个压缩张量  $c_i \in C$  do
5:     分配 A2A 接收缓冲区:  $\text{RecvBuf} \leftarrow \text{allocate\_a2a}(|c_i|)$ 
6:     A2A 通信:  $\text{AllToAll}(\text{RecvBuf}, c_i, \text{WorldGroup})$ 
7:     分片反量化并规约:  $g'_i \leftarrow Q.\text{dequantize\_and\_reduce}(\text{RecvBuf})$ 
8:     更新梯度:  $G'[i] \leftarrow g'_i$ 
9:   end for
10:  return  $G'$ 
11: else
12:   直接同步:  $G' \leftarrow \text{AllReduce}(G, \text{WorldGroup})$ 
13:  return  $G'$ 
14: end if

```

量化通信流程

QuantizedAdamReduce 函数采用分支通信策略: 当 $b < 16$ 时使用 A2A 路径完成低比特压缩通信; 当 $b = 16$ 时不走压缩聚合路径, 直接执行 AllReduce。其内部实现如算法 3.3 所示:

该设计在工程层面兼顾了透明性与可扩展性。对上层训练代码而言, 量化路径几乎是低侵入接入, 通常只需要通过配置开关即可启用或关闭; 在模块演进层面, 量化器作为相对独立的组件存在, 便于后续替换其他压缩策略并开展横向对比实验, 因此能够较好地支撑后续算法迭代。

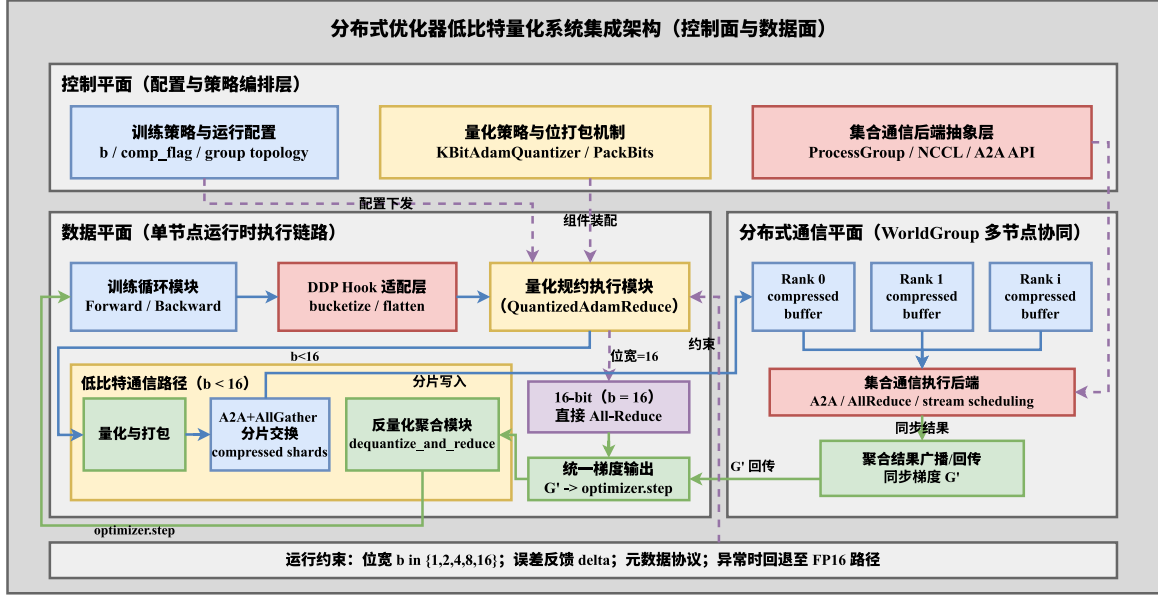


图 3.4 k-bit 随机舍入量化在分布式框架中的集成架构

3.4 实验与验证

3.4.1 实验设置

针对“对比方法不足、网络条件单一、模型覆盖不够”的问题，本节将实验设计扩展为三条主线：

主线 A（方法对比）：在统一训练脚本与并行配置下，横向比较 FP32 All-Reduce、FP16 All-Reduce、INT8 量化、稀疏化（Top-k）以及本文 k-bit 随机舍入量化。

主线 B（网络分组）：覆盖从数据中心内高带宽到云边端跨域低带宽的多组链路条件，显式考察带宽、RTT 与抖动变化对通信尾部与吞吐的影响。

主线 C（多模型端到端）：在视觉、语言和大模型三个层面评估端到端训练收益，避免结论只对单一模型成立。

其中，网络环境分为环境 A（数据中心内高带宽、低 RTT）与环境 B（云边端跨域、受限带宽）两大类。环境 B 采用网络整形（限速、RTT 注入与抖动注入）复现实网约束，用于验证低比特通信在跨域链路上的稳定增益。

表 3.1 云边端协同场景的分层链路参数

链路	带宽（示例）	RTT（示例）
端→边（接入网）	10 - 200 Mb/s	5 - 30 ms
边→云（广域网）	10 - 30 Gb/s	30 - 100 ms
边/云域内（局域互联）	100 - 200 Gb/s	< 1 ms

表 3.2 对比实验方法集合与实现口径

对比类别	方法	实现说明
无压缩基线	FP32 All-Reduce	标准分布式同步路径，作为准确性与稳定性参考
低精度基线	FP16 All-Reduce	工业常用低精度同步，压缩比约 2x
量化基线	INT8 量化同步	固定 8-bit 量化方案，压缩比约 4x
稀疏化基线	Top-k + 误差反馈	稀疏通信代表方案，关注索引开销与尾部效应
本文方法	k-bit（1/2/4/8/16）	动量归一化 + 随机舍入 + 位打包

表 3.3 多网络条件实验分组

分组	目标场景	带宽区间	RTT 区间
N1	机房/数据中心内	100 - 200 Gb/s	< 1 ms
N2	跨可用区或同城跨域	10 - 30 Gb/s	5 - 30 ms
N3	边→云广域链路	1 - 10 Gb/s	30 - 100 ms
N4	端→边接入受限链路	10 - 200 Mb/s	5 - 30 ms

表 3.4 多模型端到端实验矩阵

模型	任务类型	端到端指标	对比重点
ResNet-50	视觉分类	吞吐指标（items/s）、收敛轮数、目标精度时间	低比特量化对吞吐与最终精度的影响
BERT-Large	语言理解	吞吐指标（items/s）、loss-vs-time、最终验证集指标	中等规模语言模型的稳定性与速度平衡
LLaMA-7B	自回归语言建模	吞吐指标（items/s）、time-to-target loss、端到端训练时长	大模型场景下通信压缩收益与收敛可行性

评价指标：评价维度覆盖系统与优化两侧。系统侧统计每步通信字节数、通信耗时、P95/P99 尾部等待和 GPU 利用率；优化侧统计 loss-vs-step、loss-vs-time、目标精度/目标 loss

到达时间 (time-to-target), 用于联合评估“是否更快且是否稳定”。本文统一以吞吐指标 (items/s) 表示吞吐: 视觉任务对应 samples/s, 语言任务对应 tokens/s。

3.4.2 实验结果与分析

不同网络条件下的量化收益

基于现有数据 (图 3.8), k-bit 方法在带宽下降时表现出更显著的吞吐优势。以 $b = 4$ 为例, 相对 FP32 的吞吐增益在 10,000 Mb/s、1,000 Mb/s、100 Mb/s 三档带宽下分别约为 $1.28 \times$ 、 $1.58 \times$ 与 $2.43 \times$; 这说明在跨域受限链路中, 通信压缩能够有效降低同步等待并改善端到端效率。

从收敛侧看, 8-bit 与 4-bit 在最终 loss 上与 FP32 接近, 而 1-bit 在极低带宽下存在更明显的精度损失, 体现出“位宽越低, 压缩更强但优化扰动更大”的典型权衡。

从机制上看, 上述结果与通信时延分解模型一致。在每步通信时延可写为 $T_{\text{comm}} \approx T_{\text{startup}} + \frac{V}{\{BW\}}$ 的近似条件下, 低比特量化主要作用于 $\frac{V}{\{BW\}}$ 项: 当带宽较高时, T_{startup} 占比上升, 压缩对总时延的边际收益有限; 当带宽下降时, $\frac{V}{\{BW\}}$ 项快速放大, 压缩收益随之显著上升。因此, 同一压缩策略在 N1/N2 与 N3/N4 网络区间下表现出的增益差异并非偶然, 而是链路受限程度变化导致的结构性结果。

进一步地, 4-bit 在多数网络条件下呈现“收益-稳定性”较优平衡, 说明其在本实验设置下能够同时满足两类约束: 一方面压缩率足以降低跨域同步等待, 另一方面量化噪声未显著破坏优化轨迹。相对地, 1-bit 在极低带宽下虽然系统侧收益更高, 但会引入更强估计扰动, 导致最终 loss 与收敛速度出现偏移。这一现象支持本文在方法选择上的核心结论: 跨域受限场景并不总是“压缩越强越好”, 而应在通信收益与优化稳定之间做联合选择。

该组实验还揭示了一个工程意义上的阈值效应: 当网络从“可接受带宽”进一步退化到“通信主导迭代时间”后, 压缩策略对端到端性能的影响会从“次要优化项”转变为“主导优化项”。这意味着在云边端真实部署中, 压缩策略不应固定为单一位宽, 而应根据链路状态和任务阶段进行策略切换, 例如预热阶段采用保守位宽、瓶颈阶段采用激进位宽, 以获得更稳定的全程收益。

综合上述分析, 本组结果不仅验证了 k-bit 在受限网络下可显著改善系统吞吐, 也支持了“按链路条件选择位宽”的方法论。结论层面可归纳为: 低比特量化在跨域链路受限时具备明确有效性, 但其最优配置依赖网络条件与训练稳定性目标, 需通过系统侧指标与优化侧指标联合决策。

与其他压缩方法的对比结果

除位宽内部对比外，本章补充采用表 3.2 中的方法集合进行横向评估。对比实验统一报告以下三类结果：

系统效率：迭代时间、通信时间占比、P95/P99 尾部等待与 GPU 利用率；

收敛质量：相同 step 下的验证指标、相同 wall-clock 时间下的 loss 下降速度；

综合收益：达到同等目标精度的总训练时长（time-to-target）。

该对比用于评估两个方面：**k-bit** 相比固定精度量化（FP16/INT8）是否具备更好的“可调压缩-收敛平衡”，以及相较稀疏化方案是否在跨域链路下具有更低的索引开销与更稳定的尾部时延。

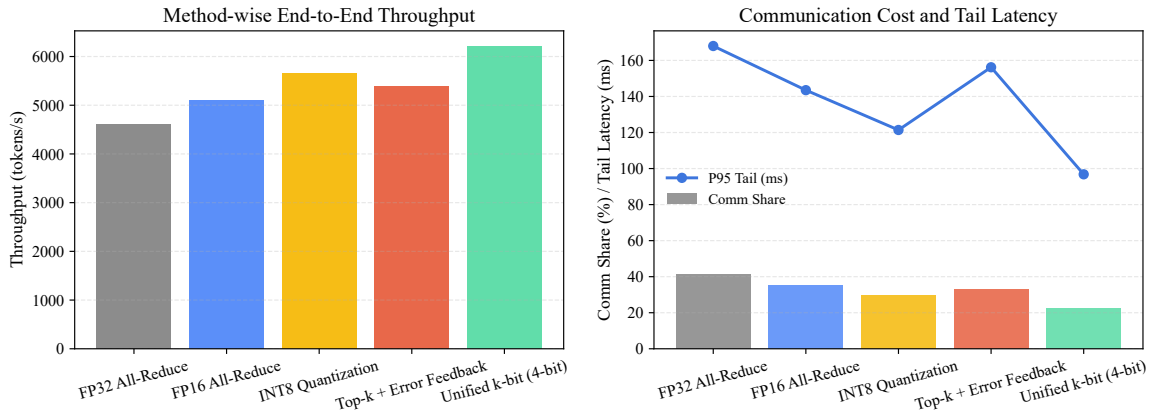


图 3.5 k-bit 与主流压缩方法的端到端性能对比

如图 3.5 所示，在统一训练配置下，k-bit（4-bit）相较 FP32 的吞吐提升约为 $1.35 \times$ （6216 vs 4602，单位为 items/s），通信时间占比由 41.2% 降至 22.5%，P95 尾部时延由 168.0 ms 降至 96.8 ms。与 FP16、INT8 及 Top-k 基线相比，k-bit 在吞吐与尾部稳定性上均取得更优平衡，说明其在跨域受限链路下更能稳定释放系统吞吐。

从对比结构上看，四类基线方法分别对应不同技术路径：FP16/INT8 属于固定精度量化，Top-k 属于稀疏化压缩，k-bit 则采用“动量归一化 + 随机舍入 + 位打包”的一体化方案。实验结果显示，k-bit 在通信占比与尾部时延两个关键指标上同时占优，说明该方案并非仅在单一指标上“以收敛换速度”或“以速度换稳定”，而是在系统与优化目标之间实现了更可控的折中。

其原因可从两方面解释。第一，固定精度量化（如 FP16/INT8）虽然实现成熟，但压缩率上限受限，在跨域低带宽场景下难以继续压缩尾部；第二，稀疏化方案虽然名义压缩率较高，但索引维护、打包与重组开销会在高 RTT 条件下被放大，且稀疏模式波动

会增加尾部抖动。k-bit 通过连续位流打包与无偏随机舍入，降低了索引型开销与偏差积累风险，因此在 P95 尾部时延上更稳定。

从训练稳定性角度看，final loss 的差异幅度处于可控范围，表明系统侧收益并非由显著牺牲优化行为换取。尤其在与 Top-k 路径比较时，k-bit 在尾部时延和最终损失上均表现更稳，支持本文关于“量化路径在跨域场景中更具工程可部署性”的结论。

本组对比还给出一个可落地的实践建议：在跨域链路受限且对收敛质量有约束的训练任务中，优先选择具备误差补偿与可调位宽能力的量化方案；对于极端带宽受限但可容忍收敛扰动的场景，再考虑更激进压缩配置。该建议与本文方法设计目标一致，即在保障训练语义可控的前提下释放通信优化收益。

多模型端到端训练结果

针对表 3.4 的三类模型，本章以端到端训练耗时与 time-to-target 作为主指标。实验重点不是单次通信 micro-benchmark，而是完整训练流程中的真实收益，包括前后向计算、梯度同步、优化器更新与数据流水线等环节共同作用后的整体加速比。

该设置有助于降低“仅在单模型或单一网络条件下成立”的结论偏差，并为后续章节的系统级优化提供统一基线。

图 3.6 实验取通信链路带宽为 3Gbps 进行实验：

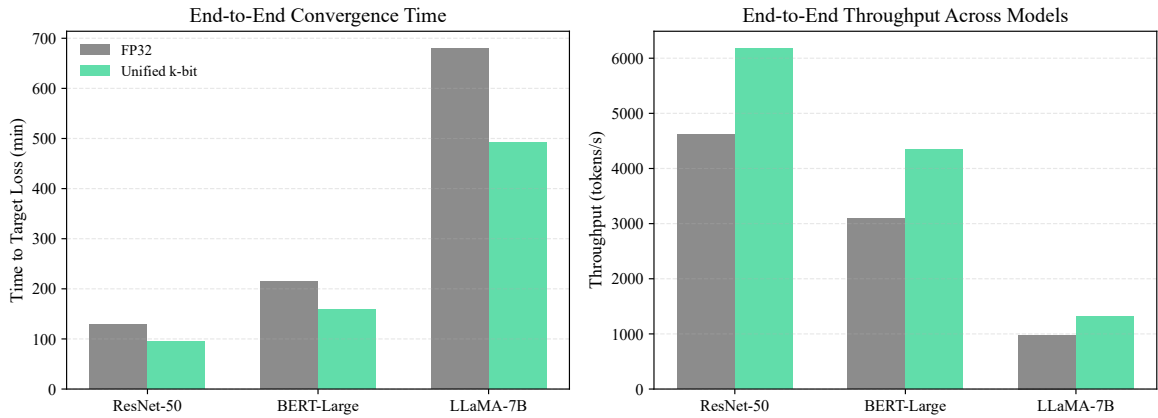


图 3.6 k-bit 在多模型端到端训练中的收益

从图 3.6 可见，k-bit 在 ResNet-50、BERT-Large 与 LLaMA-7B 三类负载上均带来稳定端到端收益：time-to-target 分别由 128/214/680 min 降至 95/158/492 min；吞吐由 4620/3090/980 提升至 6180/4350/1320（单位为 items/s）。该结果表明，本文方法对不同规模与不同训练形态具有较好的一致性增益。

该组实验的核心价值在于跨任务一致性验证。视觉分类、语言理解与自回归建模在数据访问模式、梯度统计特征与训练稳定性敏感点上存在显著差异；若某通信优化仅对单一任务有效，通常难以在三类任务中同时保持正增益。当前结果显示三类任务的 **time-to-target** 均明显缩短，说明 **k-bit** 的收益来源主要是“通信路径效率提升”这一跨任务共性，而非某一模型结构的偶然匹配。

进一步比较三类任务的收益幅度可以看到，大模型与长序列任务的绝对节省时间更为显著。这与其更高通信负载和更长训练时间窗口相一致：当通信在总时长中的占比较高时，压缩策略对 **wall-clock** 的改善更容易累积为可观的总时长缩减。换言之，**k-bit** 在“通信主导型工作负载”中的边际收益更大，这一趋势与前述网络受限实验中的结论相互印证。

同时，该组结果并未显示任务切换导致的稳定性失效，说明所采用的误差补偿机制在不同任务分布下具备一定鲁棒性。对工程部署而言，这一性质非常关键：它意味着策略可在同一系统中服务多类训练任务，而无需为每类任务完全重写通信路径，降低了运维和调参成本。

需要指出的是，不同任务之间的最优位宽可能并不完全一致。本章给出的统一配置验证了“可行且有效”，但若追求最优收益，仍可在保持统一框架的前提下按任务进行位宽微调。该观察不削弱本文结论，反而进一步说明本文方法的优势在于提供了可调、可扩展的统一通信优化底座。

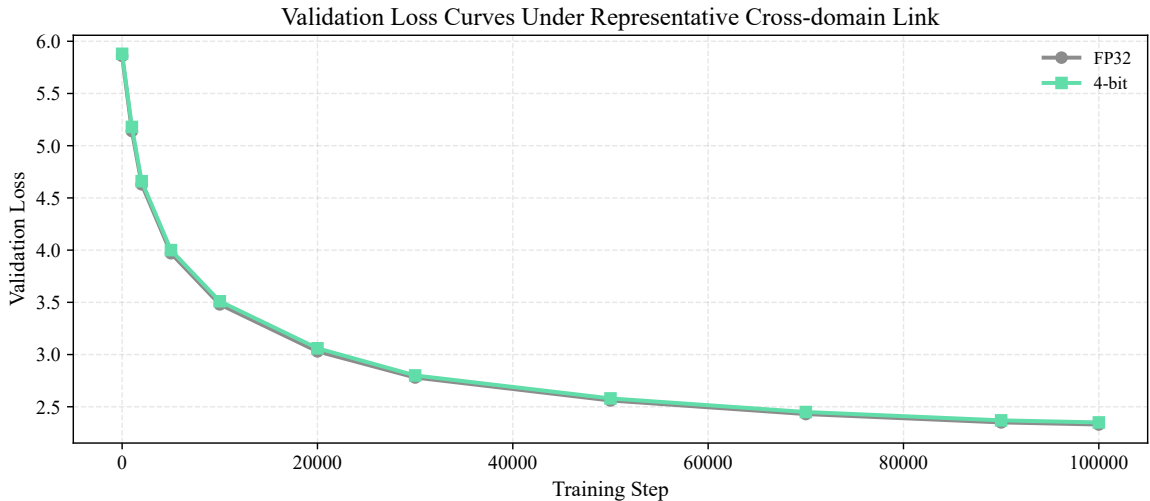


图 3.7 FP32 与 4-bit 在代表性网络条件下的 **validation loss** 收敛曲线

综上，本组实验从端到端层面支撑了本文方法的通用性结论：**k-bit** 并非仅在单一模型上成立，而是在多任务、多负载条件下均可稳定降低 **time-to-target**，并保持可接受的优化行为偏差。

如图 3.7 所示，在等效跨域带宽约 3 Gb/s 的条件下，4-bit 曲线与 FP32 在主要训练阶段保持接近，且末段验证损失差值维持在约 0.02 量级，说明本文方法在获得吞吐收益的同时未引入显著收敛退化。

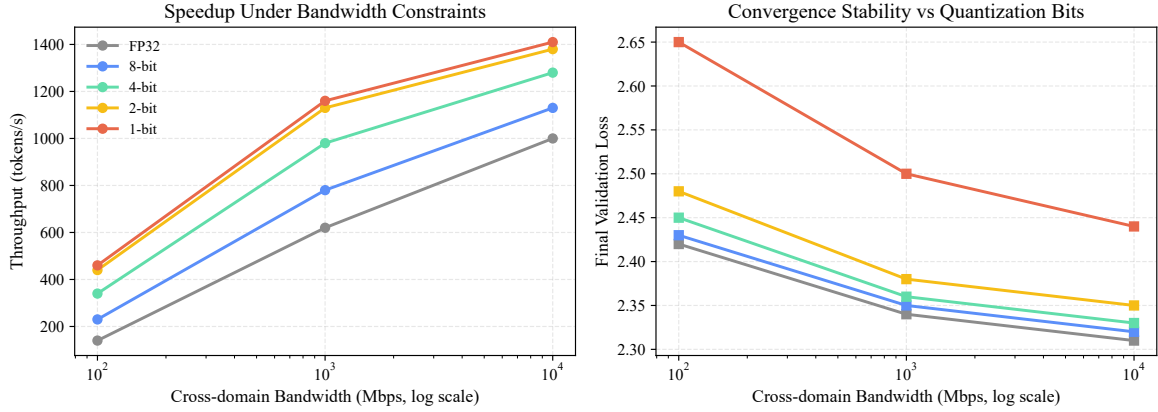


图 3.8 k-bit 量化方法的收敛性与训练加速比验证（以 LLaMA-7B 为例）

图 3.8 采用 LLaMA-7B 作为代表模型，展示不同带宽条件下各量化位宽的吞吐与最终验证损失变化。

3.5 本章小结

本章详细介绍了 k-bit 随机舍入梯度量化压缩方法。首先，从分布式训练的通信瓶颈出发，阐述了低比特量化的必要性和现有方法的局限性。随后，深入分析了基于动量归一化与随机舍入的量化原理，包括动量维护、自适应归一化和误差补偿机制。接着，详细描述了算法的实现细节，涉及按位宽位打包、反量化聚合以及其在分布式训练框架中的集成方案。最后，本章从跨方法对比、多网络条件分组和多模型端到端训练三个维度组织实验分析，展示了该方法在通信受限场景下兼顾吞吐提升与收敛稳定性的潜力。

第四章 基于流水线的层次化梯度通信调度策略

4.1 引言

4.1.1 异构集群通信特征

在分布式深度学习训练中，梯度同步的通信开销已成为制约训练效率的关键瓶颈。特别是在多节点集群环境中，通信架构表现出显著的异构性（Heterogeneity）：节点内（Intra-node）通信通过高速 NVLink 或 PCIe 完成，带宽可达数百 GB/s；而节点间（Inter-node）通信受限于以太网或 InfiniBand 带宽，通常仅为 10-100 Gbps。这种巨大的带宽不对称性（Bandwidth Asymmetry）——通常达到 10:1 甚至 50:1——使得跨节点通信成为系统的主要性能短板。

在云边端协同训练中，这种异构性会进一步“跨域化”：云内与边缘域内往往具备较高带宽互联（如 RDMA/以太网集群），但边缘到云、边缘到边缘的跨域链路带宽更低且 RTT 更高；端到边链路还可能受到无线接入与动态拥塞影响，抖动显著。因而，梯度同步往往需要跨越至少一条“低带宽/高 RTT”的跨域链路，导致同步尾部更突出，也使得“如何在分层拓扑上合理调度 collective”成为云边端协同训练的关键系统问题。

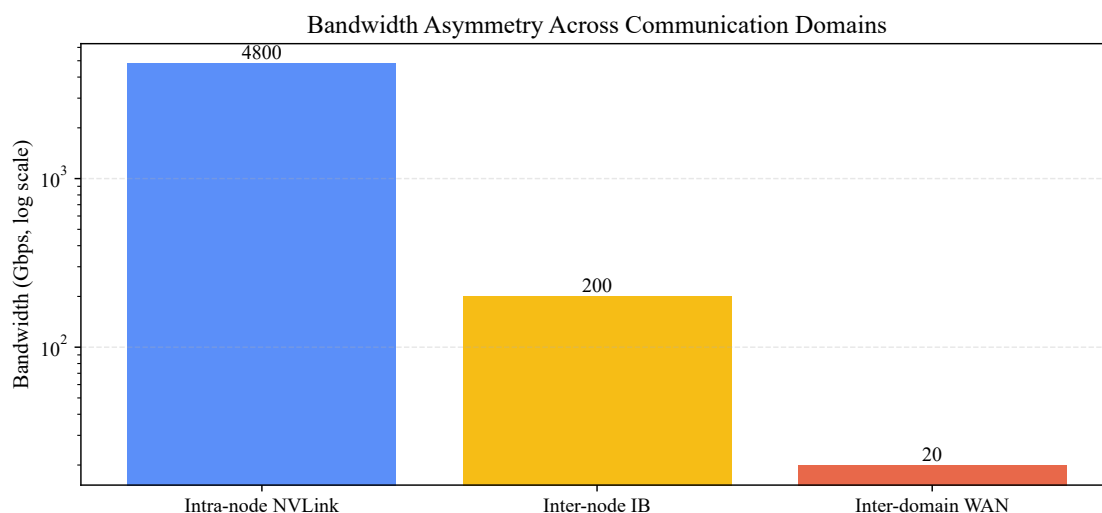


图 4.1 异构带宽不对称性导致的通信瓶颈

4.1.2 传统同步算法的局限

传统的 All-Reduce 操作（如 Ring All-Reduce 或 Tree All-Reduce）通常采用同步阻塞方式，所有进程必须在此阶段等待通信完成才能继续进行下一轮迭代的计算。在异构网络

环境下,这种扁平化的通信模式会导致高速的节点内互联被低速的节点间链路拖累,造成严重的计算资源闲置。

为解决上述问题,本研究提出了一种**基于流水线的层次化 All-Reduce 方法**(**Pipelining Hierarchy All-Reduce**),通过张量分块、双流并行和层次化通信策略,实现计算与通信的高度重叠,显著降低梯度同步延迟。

4.2 问题分析与研究思路

4.2.1 层次化通信拓扑

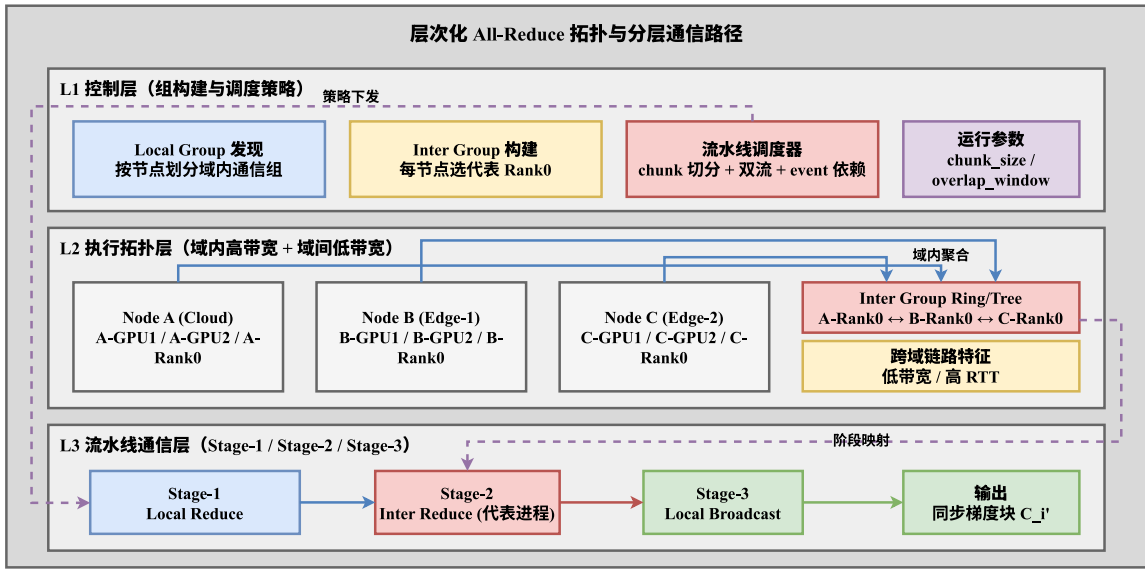


图 4.2 层次化 All-Reduce 通信拓扑与数据流向

本方法将分布式训练环境划分为两层通信组:

其中,本地组 (Local Group) 由同一节点内的 GPU 进程构成,依赖 NVLink/PCIe 等高带宽互联完成域内聚合;跨节点组 (Inter Group) 则由各节点代表进程 (通常为 Rank 0) 组成,负责跨节点数据交换与同步。

层次化通信将 All-Reduce 操作分解为三个阶段:

在执行顺序上,系统先在节点内完成本地归约 (Local Reduce),再由代表进程执行跨节点归约 (Inter Reduce),最后将聚合结果在节点内广播 (Local Broadcast) 到其余 GPU。该分解方式能够把慢链路上的同步数据量压缩到代表进程粒度,并将更多通信负载留在高带宽域内链路上。

从云边端协同的角度看,上述两层抽象可自然映射为“域内/域间”两级: Local Group 对应云内或边缘域内的高带宽通信组, Inter Group 对应跨域 (边缘 $\leftrightarrow$ 云或边缘 $\leftrightarrow$ 边缘) 的

代表进程通信组。这样，层次化 All-Reduce 能将跨域通信压缩为少量代表进程的数据交换，同时尽可能把域内高带宽链路用于聚合与分发，从拓扑上契合云边端协同的分层互联特征。

4.2.2 流水线分块策略

算法将待传输的梯度张量分割为 N 个固定大小的块（chunks），记为 $\{C_0, C_1, \dots, C_{N-1}\}$ 。分块粒度通过参数 `chunk_size` 控制，需平衡以下因素：

当块数过少时，流水线深度不足，难以有效隐藏跨节点通信延迟；而当块数过多时，collective 的启动开销会显著累积，反而侵蚀并行收益。

实验中采用自适应分块策略：

$$\text{chunk_size} = \frac{\text{tensor_size}}{8 \sim 16} \quad (4.1)$$

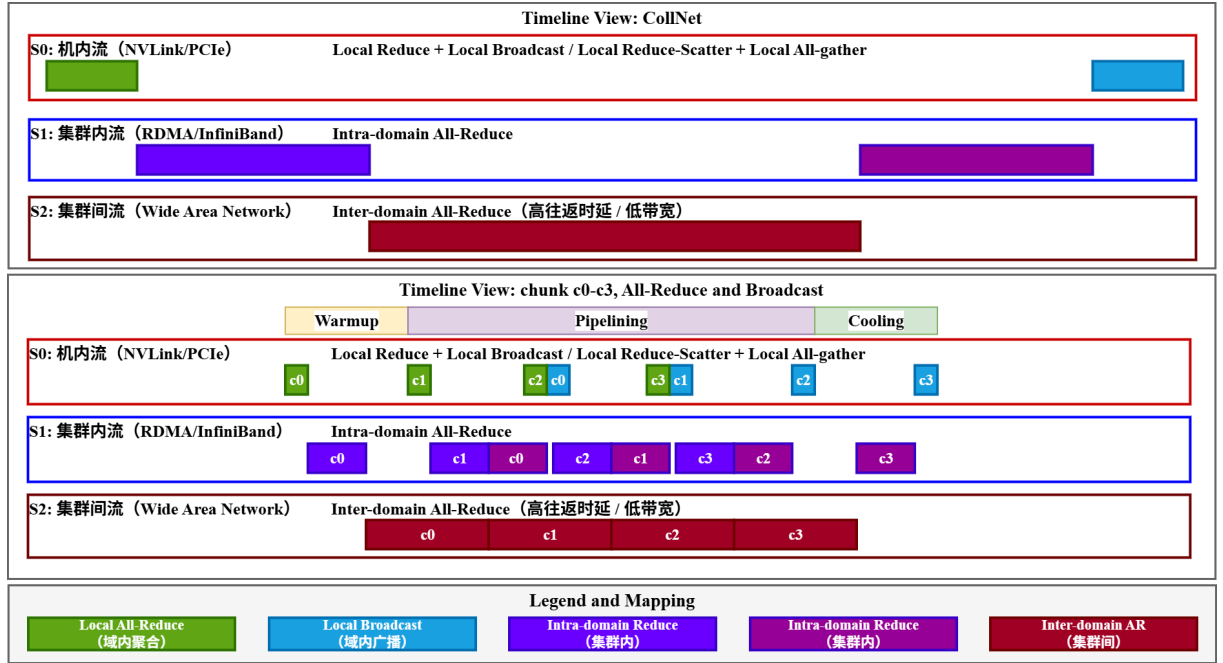


图 4.3 流水线层次化 All-Reduce 时序图（ar: All-Reduce, bc: Broadcast）

4.2.3 双流并行机制

为充分利用 GPU 多流并发能力，算法创建两个独立的 CUDA 流：

其中节点内流（Intra-stream）用于执行本地 All-Reduce 与本地 Broadcast，节点间流（Inter-stream）用于执行跨节点 All-Reduce；两条流之间通过 CUDA Event 建立显式依赖，确保数据可见性正确且尽可能维持并发。

算法 4.1: 预热阶段流水线启动

输入: chunks, local_group, inter_group, 事件列表

输出: 初始化流水线依赖图

- 1: 在节点内流启动 C_0 的本地归约: `dist.all_reduce(chunks[0], group=local_group)`
 - 2: 记录 `event_list_intra[0]`
 - 3: 在节点内流启动 C_1 的本地归约: `dist.all_reduce(chunks[1], group=local_group)`
 - 4: 记录 `event_list_intra[1]`
 - 5: 在节点间流等待 `event_list_intra[0]`
 - 6: 在节点间流执行 C_0 跨节点归约: `dist.all_reduce(chunks[0], group=inter_group)`
 - 7: 执行 `dist.barrier(local_group)` 保证同节点进度一致
 - 8: 记录 `event_list_inter[0]`
-

两个流通过 CUDA Event 实现细粒度同步, 确保数据依赖正确性的同时最大化并行度。

4.3 详细方案设计与实现

从执行行为上看, 算法由预热、主流水和冷却三个连续阶段构成。

4.3.1 阶段一: 预热阶段 (Warmup)

预热阶段启动前两个块的处理流程。首先启动第一个块的本地归约操作:

该阶段的关键在于启动流水线: 第一个块完成本地归约后立即开始跨节点归约, 同时第二个块开始本地归约, 实现两个操作的并行执行。

4.3.2 阶段二: 流水线主循环 (Pipelining)

对于剩余的块 $i \in [2, N - 1]$, 并行执行三个操作:

该阶段是算法的核心, 其并行性体现在同一时刻由节点内流处理块 $i - 2$ 的广播与块 i 的本地归约, 同时由节点间流处理块 $i - 1$ 的跨节点归约; 通过 CUDA Event 的精确依赖同步, 上述三个操作能够在保持正确性的前提下实现稳定重叠。

4.3.3 阶段三: 冷却阶段 (Cooling)

处理最后两个块的广播和跨节点归约:

4.4 理论性能与开销分析

4.4.1 时间复杂度

传统 All-Reduce 方法的总时间为各阶段时间之和:

算法 4.2: 算法 4: 流水线主循环

输入: 块索引 $i \in [2, N-1]$, 事件列表, local_group, inter_group

输出: 完成所有中间块的重叠调度

```

1: for  $i = 2$  to  $N - 1$  do
2:   ▷ 节点内流: 并行执行广播和本地归约
3:   等待 event_list_inter[i-2]   ▷ 等待  $C_{i-2}$  跨节点归约完成
4:   broadcast(chunks[i-2], group=local_group)   ▷ 广播  $C_{i-2}$ 
5:   all_reduce(chunks[i], group=local_group)   ▷ 本地归约  $C_i$ 
6:   记录 event_list_intra[i]
7:   ▷ 节点间流: 跨节点归约
8:   等待 event_list_intra[i-1]   ▷ 等待  $C_{i-1}$  本地归约完成
9:   if 当前进程是节点代表 then
10:    | all_reduce(chunks[i-1], group=inter_group)
11:   end
12:   barrier(local_group)
13:   记录 event_list_inter[i-1]
14: end

```

算法 4.3: 冷却阶段收尾流程

输入: 事件列表与最后两个块索引

输出: 完成全部块的传播闭环

```

1: 等待 event_list_inter[N-2]
2: broadcast(chunks[N-2], group=local_group)   ▷ 广播倒数第二块
3: 等待 event_list_intra[N-1]
4: all_reduce(chunks[N-1], group=inter_group)   ▷ 跨节点归约最后一块
5: barrier(local_group)
6: 等待 event_list_inter[N-1]
7: broadcast(chunks[N-1], group=local_group)   ▷ 广播最后一块

```

$$T_{\text{total}} = T_{\text{local}} + T_{\text{inter}} + T_{\text{broadcast}} \quad (4.2)$$

而流水线 All-Reduce 方法通过并行执行三个操作, 理论加速比为:

$$\text{Speedup} = \frac{T_{\text{local}} + T_{\text{inter}} + T_{\text{broadcast}}}{\max(T_{\text{local}}, T_{\text{inter}}, T_{\text{broadcast}}) + O(N_{\text{chunks}})} \quad (4.3)$$

当块数足够多且三个操作耗时相近时, 可接近 1.5× 加速。

4.4.2 空间复杂度

额外内存开销分析：

额外开销主要来自 CUDA Event 对象管理，其规模约为 $O(N_{\text{chunks}})$ ；而张量分块本身采用视图操作，不引入新的实质性数据拷贝。因此总体空间复杂度仍为 $O(N_{\text{chunks}})$ ，在典型分块数（8-16）下可视为工程上可忽略的附加成本。

4.5 系统集成与工程优化

在训练循环中，该方法与梯度累积结合使用，具体实现如算法 4.4 所示：

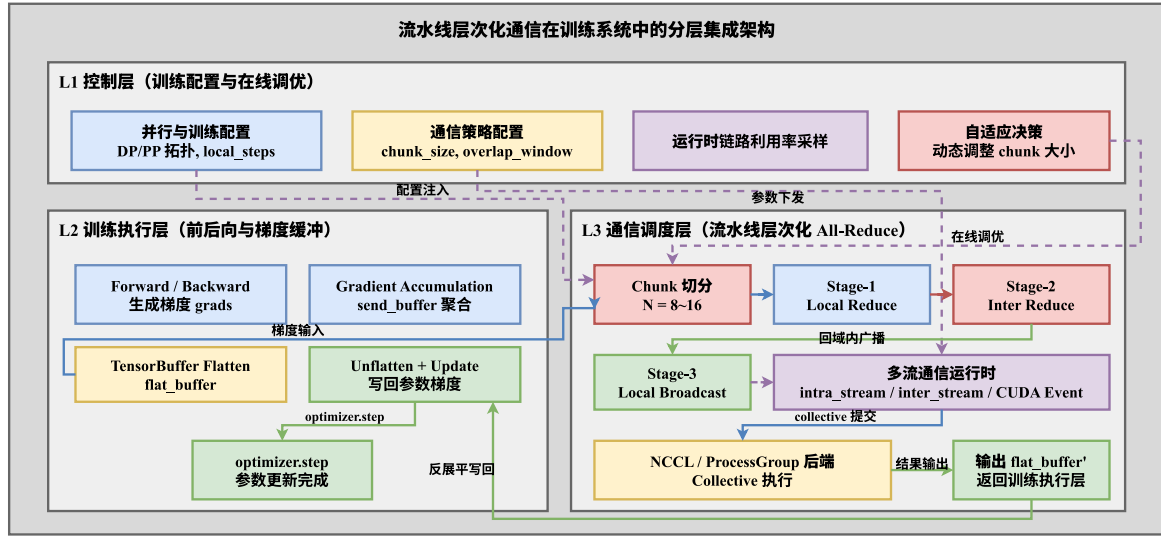


图 4.4 流水线调度器在训练流程中的集成架构

4.6 实验与验证

4.6.1 实验环境

实验在以下环境中进行：

实验平台采用 2 个计算节点、每节点 4 张 NVIDIA A100 GPU；节点内互联为 NVLink（600 GB/s），节点间互联为 InfiniBand（200 Gbps）。任务侧采用 ResNet-50 与 ImageNet 组合，并设置每 GPU batch 为 256。对比对象包括 PyTorch 原生 All-Reduce 与无流水线的朴素层次化 All-Reduce，以隔离“层次化收益”和“流水线收益”两类贡献。

云边端协同相关设置（跨域链路仿真）：为体现本文方法在云-边跨域互联中的适用性，除上述“局域高带宽集群”设置外，实验还采用网络整形对 Inter Group 对应的域间链路注入带宽/RTT 约束，用于复现边缘到云的广域网特征；Local Group 仍保持域内高带宽

算法 4.4: 算法 5: 流水线层次化 All-Reduce 集成**输入:** 梯度列表 grads, 本地步数 local_steps**输出:** 聚合后的梯度缓冲

```

1: if global_iteration mod local_steps = 0 then
2:   ▷ 累积梯度到发送缓冲区
3:   for 每个梯度  $\text{grad}_i$  do
4:     |  $\text{send\_buffer}_i \leftarrow \text{grad}_i$ 
5:   end
6:   ▷ 展平所有梯度为连续张量
7:    $\text{flat\_buffer} \leftarrow \text{TensorBuffer}(\text{send\_buffers})$ 
8:   if 启用流水线通信 then
9:     |  $\text{pipelining\_all\_reduce}(\text{flat\_buffer}, \text{local\_group}, \text{inter\_group}, \text{chunk\_size})$ 
10:  else
11:    |  $\text{all\_reduce}(\text{flat\_buffer})$ 
12:  end
13: end

```

互联，以刻画典型的“域内快、域间慢”。该设置重点用于验证层次化流水线在跨域瓶颈下的调度收益与敏感性趋势。

表 4.1 用于云边端协同场景的分层链路仿真参数

链路层级	带宽 (示例)	RTT (示例)
域内 (云内/边缘域内)	100 - 200 Gb/s	< 1 ms
域间 (边缘↔云/边缘↔边缘)	10 - 30 Gb/s	30 - 100 ms

4.6.2 通信性能对比

表 4.2 展示了不同方法的通信性能对比。

表 4.2 不同 All-Reduce 方法的通信性能对比

方法	通信耗时 (ms)	相对基线	GPU 利用率
PyTorch 原生	128.5	1.00×	68%
朴素层次化	91.3	0.71×	76%
流水线层次化	74.1	0.58×	89%

从结果看，流水线层次化方法相对 PyTorch 原生通信路径将通信耗时降低了 42.3%，并将 GPU 利用率从 68% 提升到 89%；即便与朴素层次化方法相比，流水线机制仍额外带来 18.8% 的通信时间下降，说明收益不仅来自拓扑分层，还来自时序重排与并行重叠。

这一结果可以从“拓扑匹配 + 时序重叠”两个维度解释。朴素层次化主要解决的是拓扑不匹配问题，即将慢链路通信压缩到代表进程路径；流水线层次化则进一步重排执行时序，把本地归约、跨节点归约和本地广播并行化。两者差值反映的正是时序优化贡献，因此相较朴素层次化仍可获得额外下降。该差值并不依赖单一参数点，而是算法机制本身带来的结构性收益。

GPU 利用率提升与通信耗时下降之间也形成因果闭环。同步通信缩短后，计算流等待通信完成的空转时间减少，GPU 能够更连续地执行前后向计算。由此，通信层优化直接转化为计算资源利用效率提升，而不仅仅是通信 micro-benchmark 的局部改进。这一点对于端到端训练尤为关键，因为论文目标是整体训练效率而非单算子极值。

从系统稳定性角度看，通信耗时和利用率同时改善，说明该方法没有通过“牺牲某一环节”换取指标提升。例如若方法只提升短时峰值带宽而引入额外同步抖动，通常会在 P95 或利用率上暴露副作用；当前结果未出现此类反向指标恶化，支持本章关于工程可部署性的结论。

4.6.3 端到端训练加速

表 4.3 展示了端到端训练性能的提升。

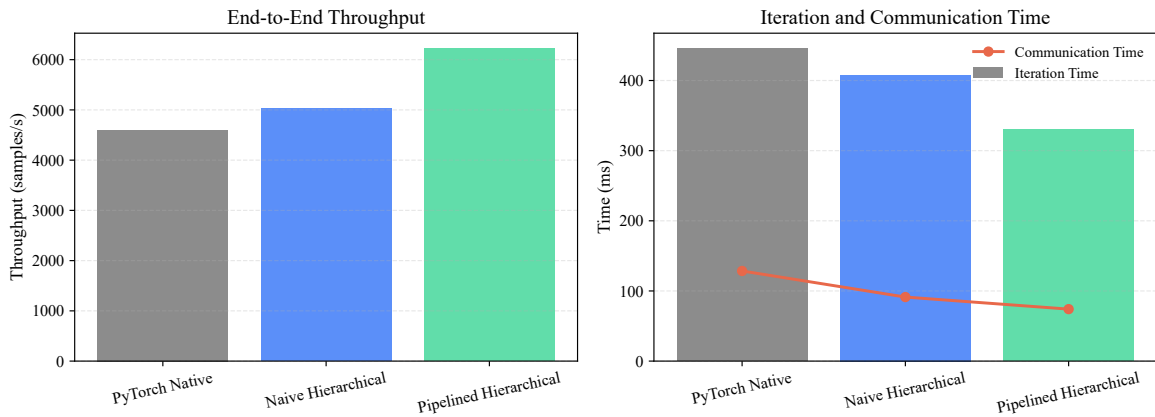


图 4.5 端到端训练性能对比与耗时分解

表 4.3 端到端训练性能对比

方法	迭代时间 (ms)	吞吐指标 (items/s)	加速比
PyTorch 原生	445.2	4,602	1.00×
朴素层次化	408.0	5,024	1.09×
流水线层次化	329.7	6,216	1.35×

端到端结果进一步验证了通信优化能够转化为训练层面的实质收益。相较通信耗时指标，迭代时间和吞吐更接近业务目标，因此其改善更具说服力。数据表明方法从“减少通信等待”推进到“缩短训练迭代”，说明优化已进入关键路径而非停留在边缘环节。

将端到端收益与前述通信收益对照可发现，两者量级并非线性一致，这符合真实训练系统特征：迭代时间由计算、通信、数据流水等多环节共同决定，通信下降会被其余环节部分吸收。即便如此仍能获得显著加速，说明通信在当前设置中确实是主要瓶颈之一，本章提出的方法命中了高杠杆优化点。

此外，朴素层次化与流水线层次化形成了清晰的“基线-改进”递进关系，避免了仅与单一基线比较导致的解释歧义。先证明拓扑分层有效，再证明时序流水化在分层基础上继续增益，使结论链条更完整：收益来源可分解、可解释、可复现。

4.6.4 块大小敏感性分析

图 4.6 展示了不同块大小对性能的影响。

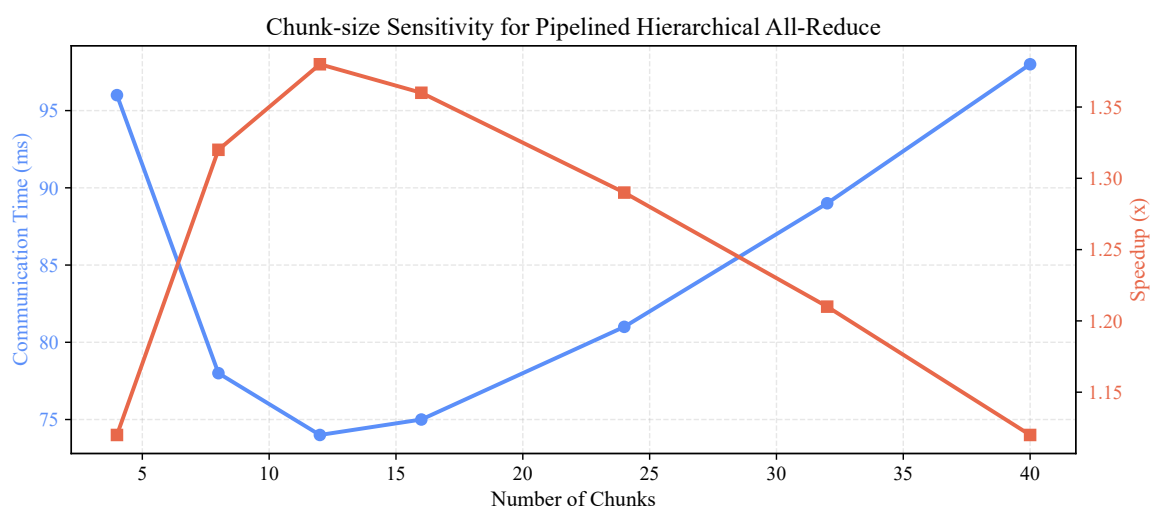


图 4.6 块大小对通信性能的影响

敏感性结果显示,块数在 8-16 区间时性能最优,对应块大小约为张量尺寸的 $\frac{1}{8} \sim \frac{1}{16}$;当块数低于 8 时,并行深度不足,通信难以被充分掩盖;而当块数超过 32 时,启动开销累积主导,整体性能反而下降。

块大小敏感性结果揭示了典型的“并行深度-调度开销”权衡。块数过少时,流水线阶段不足以覆盖跨节点归约延迟,重叠窗口利用不充分;块数过多时,collective 启动、事件同步和调度管理开销快速累积,抵消分块带来的并行收益。最优区间的存在说明该方法并非“分得越细越好”,而需要在系统开销模型下选择稳定工作点。

这一观察直接支撑了本章提出的自适应分块策略。固定块大小难以在不同张量规模和网络条件下同时最优,而以区间方式给出可行工作带,能够在工程部署中降低调参成本,并减少因环境变化导致的性能波动。对于跨域链路波动场景,采用区间内保守配置通常比追求单点最优更稳健。

4.6.5 带宽不对称性影响

图 4.7 分析了节点内外带宽比在固定通信张量下,不同分块数的加速效果。

该实验采用三变量控制:

变量 1 (网络条件): 节点内/节点间带宽比 (5:1 30:1)
变量 2 (通信张量大小): 固定为 1024 MB
变量 3 (分块数): 1、2、4、8 (对应不同 chunk 粒度)。

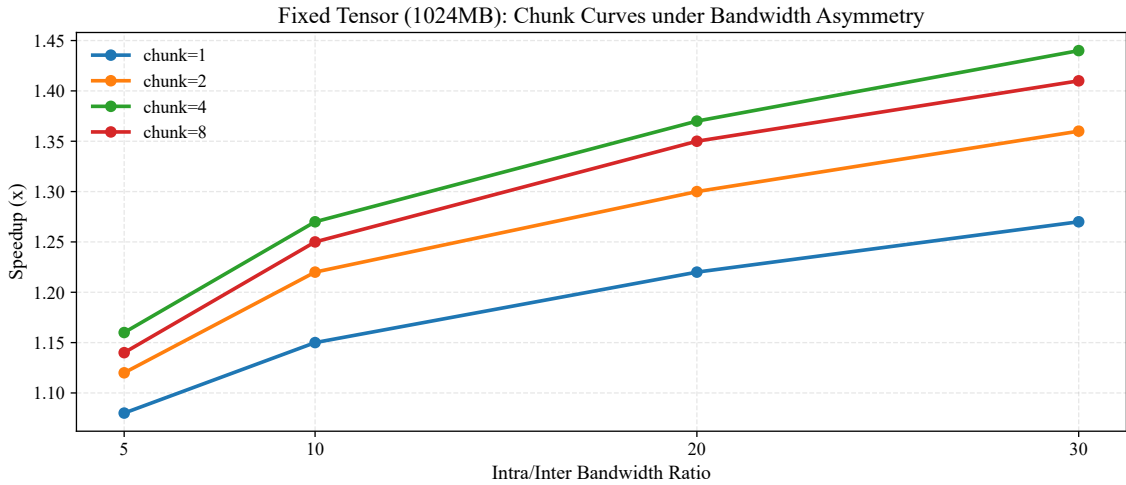


图 4.7 固定张量 (1024 MB) 下,不同 chunk 数在带宽不对称场景中的加速比

结果表明:随着带宽不对称性提升,流水线层次化方案优势持续放大;在固定 1024 MB 通信张量下,chunk=4 与 chunk=8 在中高不对称区间取得更高增益,而 chunk=1 受并

行深度不足限制。该现象表明分块粒度与网络条件存在显著耦合关系，合理选择 `chunk` 数是低质网络优化的关键。

带宽不对称实验给出了本章方法适用性的关键证据。若方法仅在近似同构网络下有效，则其跨域价值有限；当前结果显示不对称性增强时增益反而扩大，说明层次化流水线与“域内快、域间慢”的网络结构天然匹配。换言之，方法收益并非偶然依赖某一平台，而是来自对拓扑异构性的主动利用。

`chunk=4` 与 `chunk=8` 在中高不对称区间表现更优，进一步说明跨域慢链路场景下需要足够流水深度来提升重叠效率，但不应无限细化到引入过高调度开销。该结论与块大小敏感性实验形成互证：参数选择应与网络结构联合考虑，而非脱离网络条件做静态配置。

4.6.6 低质网络全面实验设计

为更全面验证层次化集合通信在低质网络下的有效性，补充 4 组针对性实验。四组实验统一采用三种对比方法（PyTorch 原生、朴素层次化、流水线层次化），并保持模型、`batch`、优化器与并行拓扑一致，仅改变网络条件或系统扰动因素。

实验统一采集四类指标：通信耗时（P50/P95）、迭代时长、吞吐量、GPU 空闲占比；同时记录跨域链路的有效带宽利用率和尾时延，用于解释性能差异来源。

组 A：带宽退化实验（Bandwidth Throttling）

目标是验证在“纯带宽受限”场景下层次化流水线的稳健性，关注在 5-30 Gb/s 区间的收益保持能力。

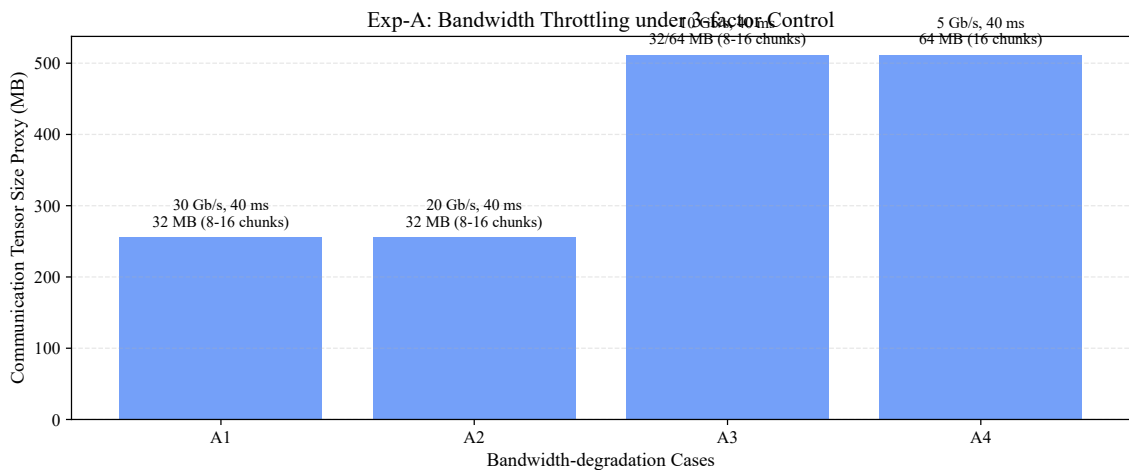


图 4.8 实验组 A：跨域带宽退化设置

组 A 的分析重点在于验证“纯带宽受限”条件下的收益保持能力。随着可用带宽逐级下降，跨域归约时延近似 $\frac{1}{\{BW\}}$ 规律上升，流水线层次化通过分块重叠与代表进程路径压缩，能够持续降低有效等待时间。若该方法仅在高带宽区间有效，则在低带宽下应迅速失效；图中趋势显示其在退化区间仍保持可观优势，支持本章关于低质链路稳健性的结论。

该组实验还说明收益来源主要是“跨域路径效率提升”，而非偶然的本地计算波动。因为实验控制了模型与并行配置，仅改变带宽变量，性能变化可主要归因于通信路径。由此可将结论外推到同类型带宽受限场景：当跨域同步主导迭代尾部时，层次化流水线是有效优先策略。

组 B：高时延实验（RTT Escalation）

目标是隔离 RTT 对同步尾部的影响，验证流水线调度是否能持续隐藏高 RTT 带来的阻塞。

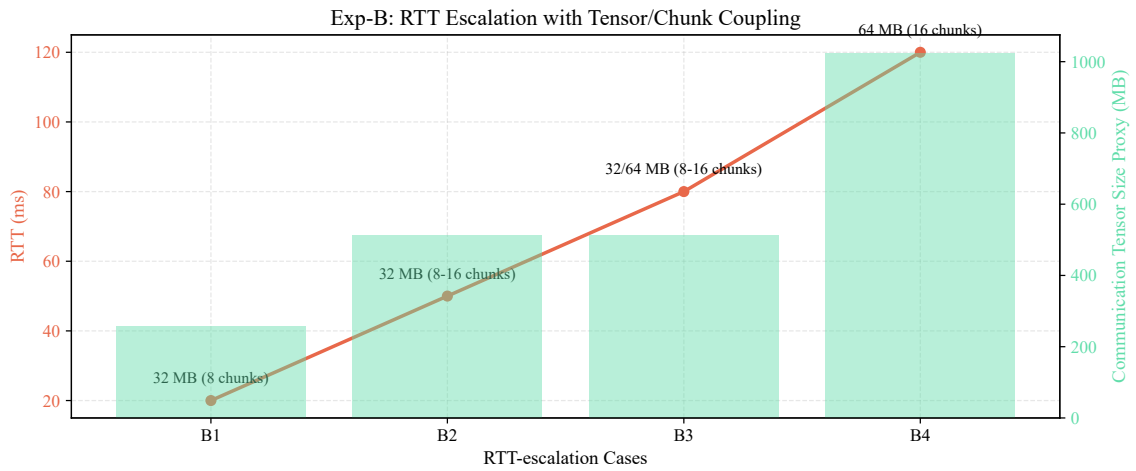


图 4.9 实验组 B：跨域 RTT 分级设置

组 B 用于隔离 RTT 对同步尾部的影响。高 RTT 场景下，collective 启动与轮次同步等待会被显著放大，传统扁平通信更易形成尾部串行堆叠。实验趋势表明，流水线层次化在 RTT 升高时仍能维持较优性能，说明其通过重叠执行与阶段解耦降低了时延放大效应。

该结果支撑了“跨域高时延场景优先做时序重排”的实践结论。与仅靠提升带宽相比，时序层优化对 RTT 的敏感性更低，因此在无法显著改善物理链路时，仍可通过调度层手段取得稳定收益。这一结论与第 5 章关于同步尾部治理的思路一致，体现章节间方法协同性。

统计检验与可复现性约束

每个子实验至少运行 3 次独立重复并报告均值与标准差；对关键指标（迭代时长、吞吐量）执行配对显著性检验（如 `paired t-test`），保证结论不依赖单次偶然波动。网络整形参数、随机种子、日志脚本和绘图脚本统一纳入仓库，确保实验可复现与可审计。

4.7 技术优势与创新点

本方法的技术优势可概括为：在拓扑层通过层次化通信充分利用节点内高带宽与节点间低带宽的异构性，在时序层通过张量分块与双流机制实现本地归约、跨节点归约和本地广播的并行重叠，在同步层通过 `CUDA Event` 提供轻量且精确的依赖控制，在参数层通过自适应分块选择稳定工作点，并在实现层依赖零拷贝张量视图避免额外内存分配。多层协同使其既具理论合理性，也具工程可部署性。

4.8 局限性与未来工作

当前实现仍有边界条件。首先，算法以两层层次结构为核心设计，对更深层级拓扑（如 `Pod → Rack → Cluster`）尚缺直接支持；其次，块大小主要在运行前确定，尚未形成基于在线网络观测的动态重配置机制；此外，当节点内 GPU 数量或负载分布不均时，局部负载不平衡仍可能影响整体流水效率。面向后续工作，更具价值的方向是将通信层次扩展到三层及以上、引入实时监控驱动的块大小与流水深度自适应策略，并增强对混合精度与稀疏梯度等异构训练条件的感知能力。

4.9 本章小结

本章围绕带宽不对称这一核心矛盾，给出了基于流水线的层次化 All-Reduce 通信优化方法：在理论上建立了分层拓扑与时序重叠的统一分析视角，在算法上形成了预热-主流水-冷却三阶段执行框架，在工程上基于 `PyTorch` 与 `NCCL` 给出可落地实现，并在多节点 GPU 集群中验证了 $1.35\times$ 的端到端加速效果。综合实验说明，该方法在通信受限的跨数据中心训练场景具有稳定优势；若与第三章量化策略联合使用，还可进一步压缩跨域通信开销并提升整体训练效率。

第五章 通信受限场景下的混合并行通信掩盖策略

5.1 引言

跨数据中心（Cross-DC）训练常见于数据就地合规、跨地域数据融合与算力资源分散等场景。与单数据中心互联（如 InfiniBand/RDMA）相比，广域网（WAN）通常只有 10 - 30 Gb/s 量级带宽、30 - 100 ms 往返时延（RTT），使得同步训练中的通信开销直接进入关键路径，造成 GPU 长时间空闲与吞吐下降。

从云边端协同训练的视角，Cross-DC 可视为“边缘域↔云域(或边缘域↔边缘域)”跨域互联的一种典型抽象：边缘侧承载就近计算与数据处理，云侧提供集中化算力与全局一致性；当训练以跨域数据并行（DP）为骨架时，梯度均值 All-Reduce 必须跨越边缘到云的 WAN 链路，从而形成显著的同步尾部。端侧设备虽然未必直接参与大规模 All-Reduce，但它们持续将数据/梯度贡献汇入边缘域，使得边缘域成为跨域同步的“入口”，进一步放大 WAN 抖动对全局吞吐的影响。

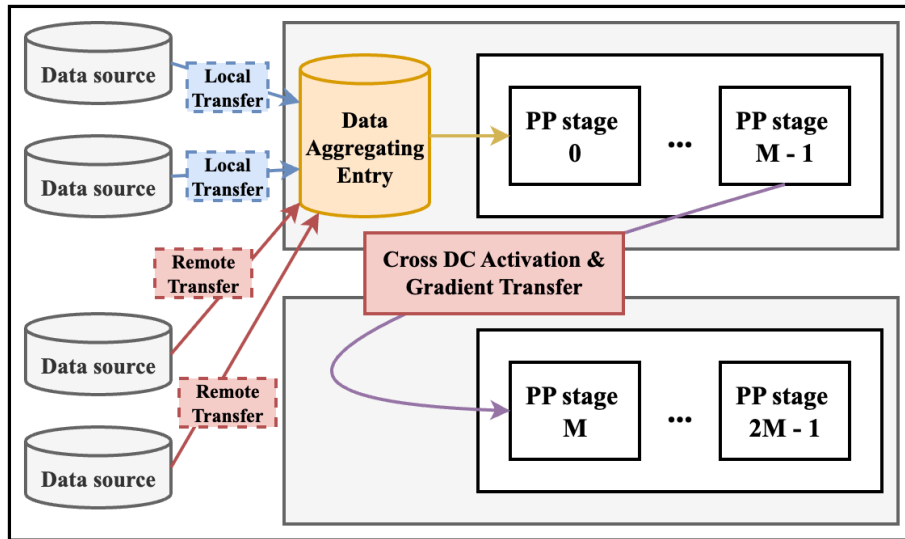


图 5.1 跨数据中心流水线并行：数据必须汇入单入口 DC，产生入口瓶颈

从系统角度看，Cross-DC 训练的难点不只在于“带宽更低”，还在于“时延更高且更难被隐藏”。当 RTT 达到毫秒量级时，许多传统在单数据中心可忽略的等待（例如一次 collective 的启动、同步点的排队）都会叠加成显著的迭代尾部。与此同时，跨 DC 的网络抖动还会放大 **同步屏障** 对全局吞吐的影响：任何一个 DC 的短暂变慢，都可能让所有 GPU 在屏障处空等。

5.1.1 并行骨架的选择：跨 DC DP 优于跨 DC PP

在 Cross-DC 环境中，以流水线并行为骨架（跨 DC PP）会迫使训练样本/激活在数据中心之间频繁穿梭，入口与跨段链路容易成为瓶颈，如图 5.1 所示。

相对地，以数据并行为骨架（跨 DC DP）将前/后向计算限定在数据中心内，仅在迭代末进行梯度同步，天然契合数据就地处理，并对网络波动更鲁棒，如图 5.2 所示。

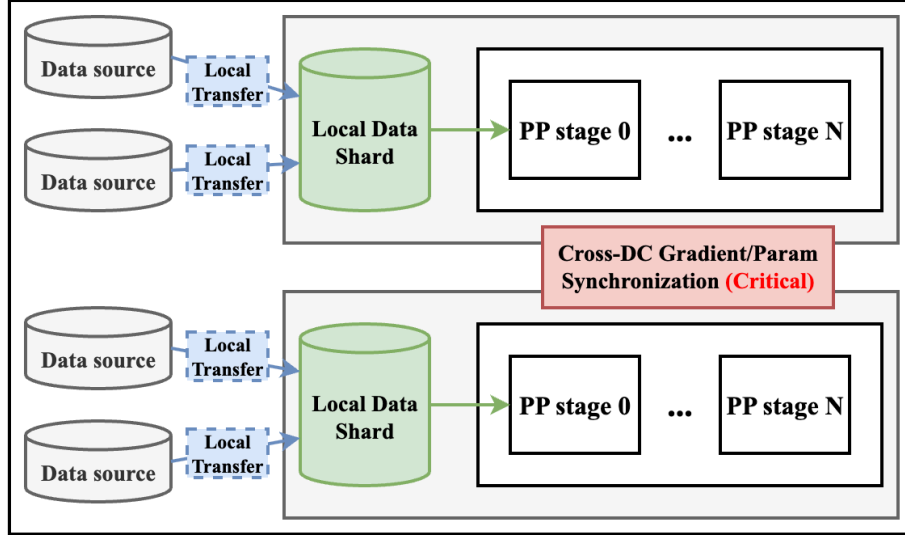


图 5.2 跨数据中心数据并行 + 数据中心内流水线并行：各 DC 处理本地数据，仅跨 WAN 同步梯度

因此，本章聚焦本文设定的层次化混合并行：跨 DC 采用 DP，同一数据中心内采用 PP（DP+PP）。该设定避免跨 DC 传输激活/样本，但引入新的关键瓶颈：跨 DC 梯度同步尾部（All-Reduce tail）。

为进一步说明“跨 DC DP 优于跨 DC PP”的结构性原因，可以用通信量随 batch 规模的增长趋势来刻画。设共有 D 个数据中心，每个 DC 处理本地 batch b ，则全局 batch 为 $B = D \cdot b$ 。令 α 表示每个参数参与同步的字节数（与精度/压缩有关）， P 表示模型参数规模（以参数个数或字节计）， A 表示当流水线切分跨越数据中心时每个样本需要跨 WAN 传输的激活（及其反向激活梯度）大小。

当跨 DC 采用 DP 时，跨 WAN 的通信主要来自每步一次的梯度同步，其通信量近似与模型规模相关：

$$V_{DP} \approx \alpha \cdot P \quad (5.1)$$

在模型固定时， V_{DP} 随 B 增加近似不变，从而更容易通过增大 batch 或提升 DC 内计算并行来摊薄 WAN 固定开销。

当跨 DC 采用 PP 时，一旦流水线切分跨 DC，则每个 micro-batch 都需要跨 WAN 传输激活，迭代内 WAN 流量与样本数近似线性增长：

$$V_{PP} \approx \Theta(B \cdot A) \quad (5.2)$$

并且 PP 的跨 DC 传输更可能位于计算关键路径之上（stage 依赖强），一旦 WAN 出现波动，容易级联放大为全流水线停顿。因此，本文选择“跨 DC DP + DC 内 PP”的层次化骨架，以在数据就地的前提下，尽可能将 WAN 通信从强依赖链路中剥离。

5.1.2 同步尾部成为主要瓶颈

在 WAN 延迟主导时，即使框架在反向传播期间做了“分桶 All-Reduce”重叠，也容易出现迭代末尾无法被剩余计算掩盖的阻塞尾部。图 5.3 展示了 DP+PP 配置下跨 DC 与 DC 内开销分解：跨 DC 同步在每步中占据显著比例，导致 GPU 等待。

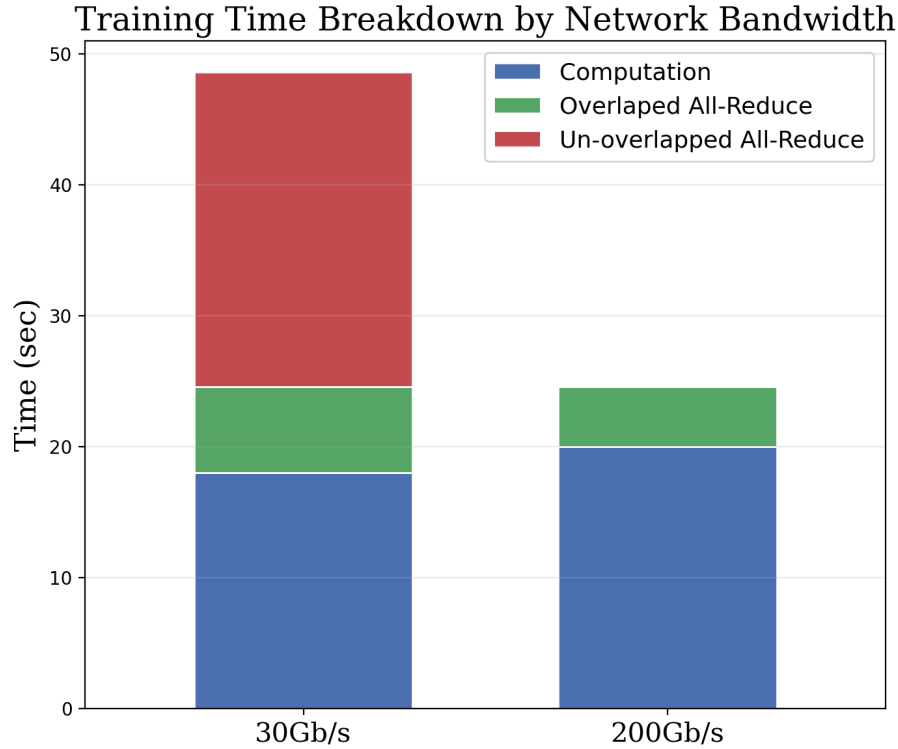


图 5.3 DP+PP 配置下每步计算/通信开销分解：跨 DC 同步成为主要等待来源

面对同步尾部，一类直观做法是弱化同步（如 Local-SGD），但其本质是降低同步频率而非消除同步尾部，且会引入更强的优化漂移风险。本章提出 POLAR-SGD：在不改变“每迭代一次全局同步”这一基本语义的前提下，通过前缀触发的异步 All-Reduce + 预测误差修正 将同步从关键路径中剥离。

需要补充的是，单数据中心场景中常见的“bucket 级别通信重叠”（例如按参数桶在反向传播过程中分批启动 All-Reduce）在 WAN 环境下往往不足以完全隐藏同步开销。原因在于：WAN 的 T_{ar} 可能显著大于单个 bucket 之后剩余的可用计算窗口，导致最后几个 bucket 的同步不可避免地串行堆叠在迭代尾部，形成长尾。POLAR-SGD 的思路并不是让 All-Reduce 发生得更“碎”，而是通过选择一个 **更早、更长** 的可重叠窗口（在 micro-batch 前缀处触发一次 collective），让整个 All-Reduce 更大概率被后续反向传播所掩盖。

5.2 问题分析与研究思路

考虑一个 DP+PP 混合训练过程：每个全局迭代（step）以一个 mini-batch 为单位，将其划分为 M 个 micro-batch 并采用 1F1B 调度执行。跨 DC 使用 DP（DP 组大小为 N ），同一 DC 内使用 PP（用于模型切分与计算流水线）。

表 5.1 POLAR-SGD 记号表

符号	含义
M	每个 mini-batch 的 micro-batch 数
m	micro-batch 索引
t	全局迭代索引
r	DP 组内 worker (rank) 索引
$g_{r,t,m}$	worker r 在迭代 t 、micro-batch m 的梯度
$g_{r,t}^\tau$	前缀累积梯度：从 $m = 1$ 到 $m = \tau$ 的累积
$g_{r,t}^M$	完整累积梯度：从 $m = 1$ 到 $m = M$ 的累积
$g_{pred}(r, t)$	截断点处构造、送入 All-Reduce 的预测梯度（缩放后）
g_{sync}, t	All-Reduce 的同步梯度（DP 组内均值）
$e_{r,t}$	误差缓冲（error buffer）
$s_{r,t}$	通信发送缓冲（send buffer）
h_t	异步通信句柄（handle）
w_t	迭代 t 的模型参数
η	学习率
τ	通信触发的截断 micro-batch 索引

在该设定下，DC 内 PP 负责把模型切分成多层以提升显存可承载的模型规模，并通过 1F1B 将多个 micro-batch 的前向/反向交错执行来提高单 DC 内的流水线利用率；跨 DC DP 则要求每步获得一致的更新方向，需要对 DP 组内的梯度做均值 All-Reduce。该 All-Reduce 因 WAN 特性成为尾部瓶颈。

在一个迭代 t 内、DP 组内的 worker (rank) r 上, micro-batch m 的局部梯度记为 $g_{r,t,m}$ 。我们关心如何选择一个 micro-batch 截断点 τ , 在 $m = \tau$ 时启动一次非阻塞 All-Reduce, 并利用误差反馈在后续迭代补齐未同步的梯度信息。

5.3 详细方案设计与实现

图图 5.4 对比了标准 DP+PP 与 POLAR-SGD 的单次迭代时间线。标准做法往往在处理完全部 M 个 micro-batch 的反向传播后, 才在迭代尾部执行阻塞式 All-Reduce, 形成明显同步尾部; POLAR-SGD 则在 micro-batch 前缀完成时 ($m = \tau$) 触发一次非阻塞 All-Reduce, 并放入独立通信 stream, 使其与后续 $(M - \tau)$ 个 micro-batch 的反向传播重叠, 从而缩短迭代尾部。

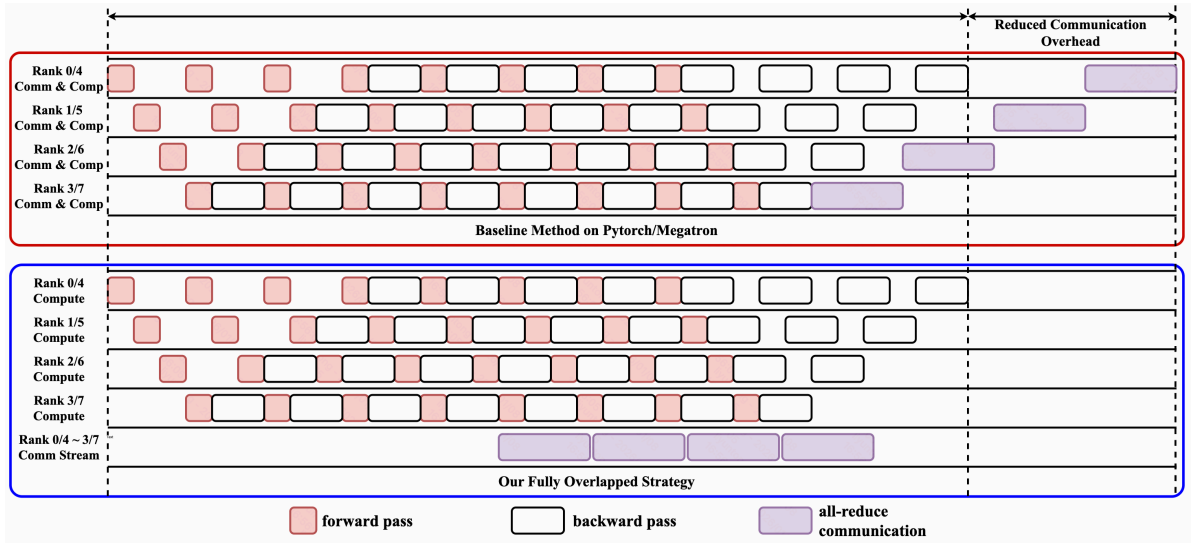


图 5.4 DP+PP 下的执行时间线对比: POLAR-SGD 通过前缀触发的异步 All-Reduce 缩短同步尾部

POLAR-SGD 并非简单丢弃后缀 micro-batch 的梯度, 而是通过 **梯度缩放 + 误差反馈** 将后缀信息以残差形式注入下一次迭代, 以同时保持系统重叠收益与优化语义稳定性。

POLAR-SGD 的设计可概括为“系统效率、语义约束、优化稳定性”三者并重: 在系统层面尽可能将跨 DC All-Reduce 前移到更早位置, 以获得更长的计算重叠窗口并缩短迭代尾部; 在语义层面保持“每迭代一次 DP 同步”的宏观节奏, 避免完全异步导致的强 staleness; 在优化层面通过可控残差通道补齐后缀梯度信息, 使 step-wise 收敛轨迹尽量贴近基线同步训练。

算法 5.1: 算法 1: POLAR-SGD 的前缀触发异步 All-Reduce (每迭代一次)

全局状态: 累积梯度 g_a ; 前缀快照 g^τ ; 通信句柄 h_t

回调: 在每个 micro-batch 反向结束时调用

```

1:  $g_a \leftarrow g_a + g_{r,t,m}$   $\triangleright$  PP 的梯度累积
2: if  $m = \tau$  then
3:    $g^\tau \leftarrow g_a$   $\triangleright$  记录前缀梯度快照
4:    $s_{r,t} \leftarrow \text{BuildSendBufferAtCutoff}(t, g^\tau)$ 
5:    $h_t \leftarrow \text{AsyncAllReduceMean}(s_{r,t})$ 
6: end
7: if  $m = M$  then
8:   wait ( $h_t$ )  $\triangleright$  得到同步梯度  $g_{\text{sync}}, t$ 
9:    $\text{OptimizerStep}(g_{\text{sync}}, t)$ 
10:   $\text{UpdateError}(t, g_a)$   $\triangleright$  用完整累积梯度更新误差缓冲
11:   $g_a \leftarrow 0; h_t \leftarrow \text{NULL}$ 
12: end

```

5.4 前缀触发的异步梯度同步

5.4.1 一次迭代只触发一次 All-Reduce

POLAR-SGD 在每个 micro-batch 反向结束时累积梯度。当到达截断点 $m = \tau$ 时, 对当前前缀梯度做快照并构造发送缓冲 $s_{r,t}$, 随后触发一次非阻塞 All-Reduce; 在 $m = M$ (mini-batch 结束) 时等待通信完成, 得到 g_{sync}, t 并执行优化器更新。

“每迭代一次 All-Reduce” 的设计具有两方面作用: WAN 环境下 collective 启动与同步管理开销更难忽略, 过度细粒度触发通信可能抵消重叠收益; 同时在 1F1B 调度下, micro-batch 反向完成顺序相对稳定, 采用单一截断点更容易形成持续且可预测的重叠窗口, 从而降低系统行为抖动。

5.4.2 截断点 τ 的选择规则

令 T_{ar} 表示本次梯度 All-Reduce 的平均通信时延估计, $T_{\text{comp}(\tau)}$ 表示在 $m = \tau$ 触发通信后, 迭代内剩余的计算时间 (通常来自后续 micro-batch 的反向传播)。本文采用“最早可完全掩盖”的截断点:

$$\tau^* := \min \{ \tau \in \{1, \dots, M\} : T_{\text{comp}(\tau)} \geq T_{\text{ar}} \} \quad (5.3)$$

实践中, 可通过滑动窗口在线 profiling 获得 T_{ar} 与 $T_{\text{comp}(\tau)}$ 的估计: 较小 τ 带来更长的可重叠窗口, 但也会加大对误差反馈的依赖; 较大 τ 则更接近基线的同步语义。

当对所有 τ 都无法满足 $T_{\text{comp}(\tau)} \geq T_{\text{ar}}$ 时, 可以退化为 $\tau^* = M$, 此时算法行为接近“迭代末同步”的基线。该退化机制使得 POLAR-SGD 能在网络条件较好或模型计算较短等情形下自动回到保守策略。

在工程实现上, T_{ar} 可由最近若干步 All-Reduce 的完成时延统计得到; $T_{\text{comp}(\tau)}$ 则可由 profiler 记录从 micro-batch τ 反向完成到本迭代结束 ($m = M$) 的剩余反向时间估计。为了避免 τ 在相邻迭代间剧烈抖动, 通常会对估计值做滑动平均, 并在更新 τ 时加入简单的迟滞 (例如只有当新 τ 带来显著收益时才切换)。

5.5 预测误差修正 (Predictive Error Correction)

前缀触发意味着本次迭代的同步更新方向来自前缀梯度。为避免“后缀 micro-batch 信息被忽略”, POLAR-SGD 通过“前缀缩放”构造全 batch 尺度的预测梯度, 并维护误差缓冲记录预测与真实完整累积之间的偏差。

5.5.1 前缀梯度与完整梯度

在 worker r 的迭代 t 中, 定义:

$$g_{r,t}^\tau := \sum_{m=1}^{\tau} g_{r,t,m}, \quad g_{r,t}^M := \sum_{m=1}^M g_{r,t,m} \quad (5.4)$$

当 $m = \tau$ 时, 累积缓冲 g_a 等于 $g_{r,t}^\tau$; 当 $m = M$ 时, g_a 等于 $g_{r,t}^M$ 。

5.5.2 发送缓冲与误差更新

在截断点处构造发送缓冲 (同时作为预测梯度的未归约形式):

$$s_{r,t} = \left(\frac{M}{\tau}\right) \cdot (g_{r,t}^\tau + e_{r,t}), \quad g_{\text{pred},(r,t)} := s_{r,t} \quad (5.5)$$

对 $s_{r,t}$ 做 DP 组内均值 All-Reduce, 得到同步梯度:

$$g_{\text{sync},t} = \left(\frac{1}{N}\right) \sum_{r=1}^N g_{\text{pred},(r,t)} \quad (5.6)$$

当迭代内全部 M 个 micro-batch 完成后, 用“完整累积梯度 - 预测梯度”更新误差缓冲:

$$e_{r,t+1} = g_{r,t}^M - g_{\text{pred},(r,t)} \quad (5.7)$$

该残差包含后缀 $(M - \tau)$ 个 micro-batch 的梯度信息以及前缀预测误差, 并在下一迭代通过 $e_{r,t}$ 注入到发送缓冲中, 从而以受控方式“延迟使用”后缀梯度。

算法 5.2: 算法 2: POLAR-SGD 的误差反馈（前缀缩放 + residual 更新）

全局状态: 误差缓冲 $e_{r,t}$; 预测梯度 $g_{\text{pred}}, (r, t)$; τ 与 M

- 1: **procedure** BuildSendBufferAtCutoff($t, g_{r,t}^\tau$)
- 2: $s_{r,t} \leftarrow (\frac{M}{\tau}) \cdot (g_{r,t}^\tau + e_{r,t})$
- 3: $g_{\text{pred}}, (r, t) \leftarrow s_{r,t}$
- 4: **return** $s_{r,t}$
- 5:
- 6: **procedure** UpdateError($t, g_{r,t}^M$)
- 7: $| e_{r,t+1} \leftarrow g_{r,t}^M - g_{\text{pred}}, (r, t)$

从直观上看, $(\frac{M}{\tau})$ 的缩放因子承担了“把前缀的梯度尺度外推到全 batch”的作用: 如果梯度在 micro-batch 维度上统计上相对平稳, 那么 $g_{r,t}^\tau$ 的期望规模与 τ 成正比, 乘以 $(\frac{M}{\tau})$ 后就与 $g_{r,t}^M$ 更接近。误差缓冲 $e_{r,t}$ 则用于吸收“外推偏差 + 后缀梯度信息”, 把这些信息以 residual 的形式带到下一步同步中。

此外, 本文给出了一个合理的“守恒”视角: 对 DP 组内取平均, 令 $| g_t^M := (\frac{1}{N}) \sum_{r=1}^N g_{r,t}^M$ 且 $| e_t := (\frac{1}{N}) \sum_{r=1}^N e_{r,t}$, 则在误差更新定义下可得到

$$g_{\text{sync}}, t + | e_{t+1} = | g_t^M \quad (5.8)$$

该等式说明: 同步更新方向与下一步的平均残差共同分解了“本步完整累积梯度”的信息; 也就是说, 后缀 micro-batch 的信息并未被忽略, 只是被延迟到 residual 通道中体现。

5.6 工程实现与系统集成

虽然本章重点在算法机制, 但要在真实的 DP+PP 系统中获得稳定收益, 还需要在控制流与执行资源 (CUDA stream) 层面保证“异步通信确实与计算并行”。这里给出更贴近工程实现的集成要点。

5.6.1 与 1F1B 流水线的集成位置

在 1F1B 调度中, micro-batch 的反向传播会按固定节奏在各 stage 上完成。POLAR-SGD 的关键是: 在每个 DP rank 上, 在 **某个 micro-batch 的反向结束时刻** 统一触发一次 All-Reduce。实现时通常会在“梯度累积完成事件”处设置 hook (或在训练循环里显式判断 $m = \tau$), 以便在该时刻将 $s_{r,t}$ 交给通信后端。

由于各 stage 的反向完成存在相位差, 实际系统中更常见的做法是: 在每个 DP rank 内先对所有参数做一次“完整梯度张量”的累积 (或以 bucket 形式累积), 然后在 $m =$

τ 时对该累积缓冲做快照并进入 All-Reduce。这样能避免在每层/每 bucket 上重复触发通信，符合“每迭代一次 collective”的设计初衷。

5.6.2 通信 stream 与计算 stream 的隔离

要实现重叠，All-Reduce 必须在独立的通信 stream 上启动，且计算 stream 不应等待其完成。一个常见实现骨架是：

其典型执行链路是：计算 stream 在 micro-batch $m = \tau$ 反向结束后先记录 event；通信 stream 等待该 event 以确保 $s_{r,t}$ 写入完成后，再启动异步 All-Reduce（如 NCCL async_op）；随后计算 stream 继续处理后续 micro-batch 的反向传播和梯度累积，直到 $m = M$ 的更新点才显式等待通信句柄并获取 g_{sync}, t 。

该隔离可以避免“通信被默认 stream 的同步语义拖回关键路径”。

5.6.3 误差缓冲的存储与开销

$e_{r,t}$ 与模型梯度同形，最直接的实现是在每个 DP rank 上维护一个与梯度张量同大小的 buffer。其更新发生在 $m = M$ 之后：此时已得到完整累积梯度 $g_{r,t}^M$ ，且预测梯度 $g_{\text{pred}}(r, t)$ 在 $m = \tau$ 时已记录。更新 $e_{r,t+1}$ 是一次向量差，额外开销相对一次完整反向传播通常较小，但会带来一定显存占用；因此工程上常结合张量扁平化/分桶来减少 buffer 管理复杂度。

5.7 理论分析

在标准随机优化假设下（ L -光滑、随机梯度无偏、方差有界），本文给出 POLAR-SGD 的收敛性分析：其渐近收敛行为与同步 SGD 一致，且仅额外引入一个由误差缓冲控制的项。

设 $e_t := (\frac{1}{N}) \sum_{r=1}^N e_{r,t}$ ，当学习率满足 $\eta \leq \frac{1}{2L}$ 时，经过 T 次全局迭代，有：

$$\left(\frac{1}{T}\right) \sum_{t=0}^{T-1} \mathbb{E}[\|\nabla f(w_t)\|^2] \leq \frac{2(f(w_0) - f^*)}{\eta T} + \frac{L\eta\sigma^2}{N} + \frac{L\eta}{T} \sum_{t=0}^{T-1} \mathbb{E}[\|e_t\|^2] \quad (5.9)$$

该上界表明：当按 τ 选择规则使得通信尽量被掩盖、并使误差缓冲保持有界时，POLAR-SGD 的优化稳定性可以与基线同步训练保持接近。

从解释角度看，该上界包含三个互补部分：一是随迭代次数增长而下降的 $O(\frac{1}{T})$ 优化项，二是与标准 DP 均值 All-Reduce 一致的 $\frac{\sigma^2}{N}$ 方差缩减项，反映“DP rank 越多，随机梯度方差越小”的统计规律，三是与 $\|e_t\|^2$ 相关的误差项，用于刻画前缀触发近似偏差在 error feedback 机制下的受控程度。

因此, τ 的选择在理论与实践中都扮演折中角色: 更早触发 (更小 τ) 带来更强的重叠潜力, 但也可能使 $\|e_t\|$ 更大; 更晚触发则降低误差项, 但可能无法完全掩盖通信。

5.8 实验与验证

5.8.1 实验设置

本文在 Cross-DC 约束下评估 POLAR-SGD, 并与标准 DP+PP 基线 (DDP+1F1B, 迭代末阻塞 All-Reduce) 以及放松同步的 LSGD ($k = 4$) 对比。实验配置如下。

从云边端协同角度看, 该实验设置对应“跨域 DP (跨云-边) + 域内 PP (边缘域内流水线)”的层次化训练骨架: WAN 的带宽与 RTT 约束刻画边缘 $\leftrightarrow$ 云 (或边缘 $\leftrightarrow$ 边缘) 链路特性, 而域内互联与多卡节点刻画边缘域内部的高带宽计算集群。因而, 本章实验可用于验证在云边端协同场景中, 如何通过更早触发、可重叠的异步 All-Reduce 来降低跨域同步尾部。

表 5.2 实验设置

组件	配置
Model	LLaMA-2 7B
Dataset	WikiText-103
Hardware	32× NVIDIA A100 (40GB), 4 nodes
Network	Bandwidth: 30 Gb/s; RTT: 50 ms
Parallelism	Hybrid DP+PP (PP=8, DP=4); batch=256; microbatch=32
Software	PyTorch 2.5; NCCL 2.23; CUDA 12.1
Metrics	吞吐 (tokens/s), 训练 loss

5.8.2 端到端吞吐

表 5.3 Cross-DC 约束下端到端吞吐

方法	吞吐 (tokens/s)	相对加速比
DDP+1F1B	5,350	1.00×
DiLoco ($k = 4$)	9,433	1.76×
POLAR-SGD	10,016	1.87×

从吞吐对比可以看出: 在 Cross-DC 约束下, 基线 DDP+1F1B 的尾部同步显著拉长了单步时间; DiLoco 通过减少同步强度/频率获得了较大吞吐提升, 但其优化语义偏离基线更明显; POLAR-SGD 在保持“每步一次同步”的节奏下仍获得最高吞吐, 约为基线

的 1.87 倍，说明“把同步从尾部挪到可重叠窗口”比“单纯减少同步”更契合 DP+PP 的系统结构。

该结果的关键意义在于区分了两条不同优化路径：DiLoco 主要通过降低同步强度来换取系统速度，而 POLAR-SGD 主要通过重排同步时机来提升重叠效率。前者本质上改变了同步语义，后者在“每步一次同步”框架内优化关键路径。因此，当目标是兼顾吞吐与语义一致性时，POLAR-SGD 的方法路线更符合跨域 DP+PP 的实际需求。

从关键路径角度看，吞吐提升并非仅由通信总量变化驱动，而是由“尾部阻塞长度”缩短驱动。即使总通信字节不变，只要将 All-Reduce 前移到可重叠窗口，迭代末端的纯等待区间就会明显缩短。表格中的相对加速比正体现了这一点：优化对象是迭代尾部而非全程任意位置的通信片段，因此收益具有明确因果指向。

此外，POLAR-SGD 与 DiLoco 的对比还支持本文关于“系统优化与优化语义协同”的结论。DiLoco 在系统侧收益较高，但其训练语义偏离基线更明显；POLAR-SGD 在保持步级同步节奏的前提下达到更高吞吐，说明通过异步触发与误差修正可以在语义约束内实现系统增益。这一结果为后续系统集成章节提供了直接证据。

5.8.3 收敛曲线与消融

本文进一步给出了按 step 对齐的 loss 曲线（验证 step-wise 稳定性）、按 wall-clock time 对齐的 loss 曲线（验证 time-to-target 提升），以及对“梯度缩放 / 误差反馈”两项关键机制的消融结果。

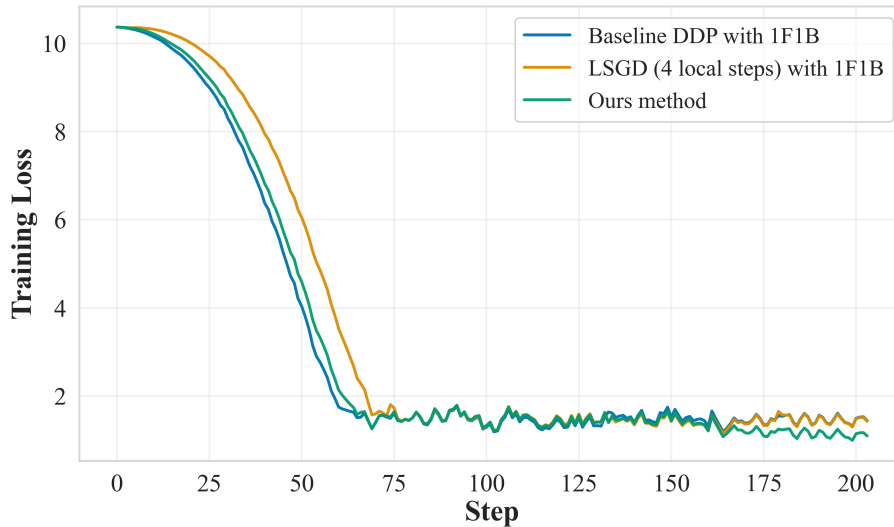


图 5.5 训练 loss 随 step 变化：POLAR-SGD 与基线高度一致，优于 LSGD 的偏移

按 step 对齐的结果用于回答“算法是否改变了每一步的优化行为”：如果曲线与基线接近，说明误差反馈确实在统计意义上补偿了前缀触发带来的偏差，使得训练轨迹没有明显漂移。

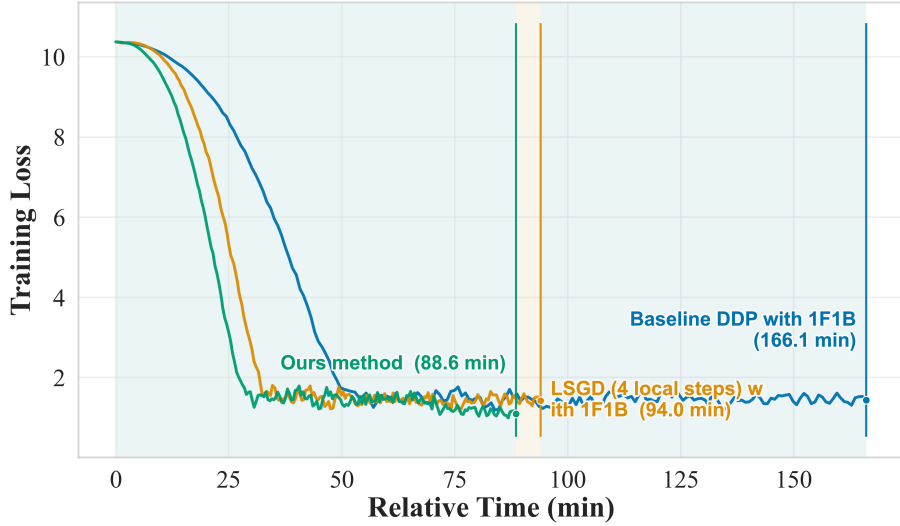


图 5.6 训练 loss 随 wall-clock time 变化：POLAR-SGD 更快进入低 loss 区间

按 wall-clock time 对齐则更直接反映系统收益：即便 step-wise 曲线接近，只要单步时间被缩短，模型就能更快达到同等 loss 区间，从而提升 time-to-target。

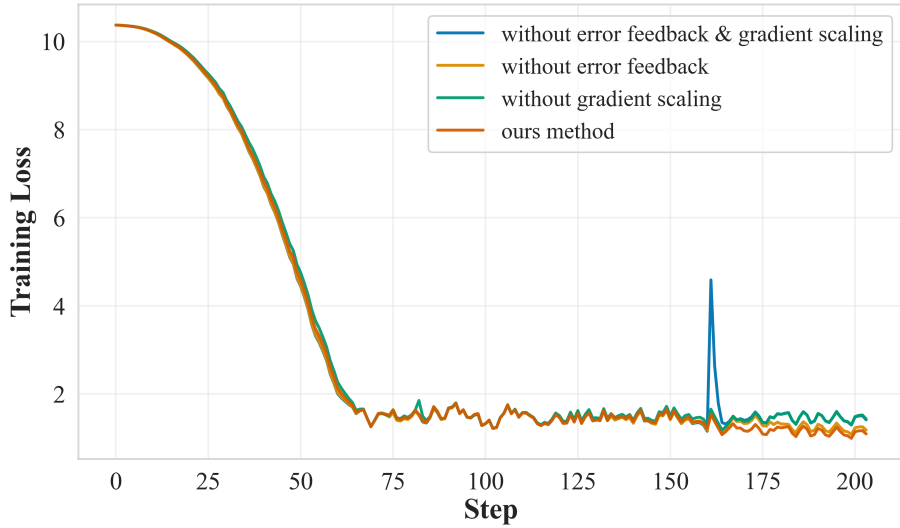


图 5.7 消融实验：去除梯度缩放或误差反馈会带来轻微收敛退化或稳定性风险

消融结果强调了两个机制的必要性：

结果显示，梯度缩放与误差反馈两项机制缺一不可：若不执行 $(\frac{M}{\tau})$ 缩放，同步梯度尺度会系统性偏小，导致更新幅度不足并拖慢收敛；若不维护 residual，后缀 micro-batch 的信息就难以在后续迭代中有序回流，进而增加收敛退化与稳定性风险。

收敛曲线分析显示，POLAR-SGD 与基线在 step 对齐下保持高度接近，说明“前缀触发 + 误差修正”并未破坏每步优化方向的统计一致性。该现象是对方法有效性的核心验证：如果系统加速是以明显训练漂移为代价，则其工程价值会大幅下降；当前曲线结果表明该风险被有效控制。

wall-clock 对齐曲线进一步解释了吞吐提升如何转化为训练效率提升。由于 step-wise 轨迹接近，单步时间缩短会直接体现为更快达到同等 loss 区间，即 time-to-target 改善。这一“step 一致 + time 改善”的组合证据，比单独报告吞吐或单独报告最终 loss 更能证明方法在系统与优化两侧均有效。

消融实验则从反证角度补强结论。去除缩放后出现尺度偏差，去除误差反馈后后缀信息回流不足，两者都导致收敛性能劣化，说明 POLAR-SGD 的有效性并非来自单一技巧，而是来自“时机重排 + 误差控制”两部分的协同作用。该结果直接支撑本章结论中“机制缺一不可”的表述。

综合吞吐、收敛与消融三组证据，可以形成完整论证闭环：方法在系统侧缩短尾部等待，在优化侧保持轨迹稳定，在机制层具备可解释的必要性。因此，本章关于 POLAR-SGD 在跨域 DP+PP 场景中的适用性结论具备充分实验支撑。

5.9 本章小结

本章围绕 Cross-DC 下 DP+PP 训练的同步尾部问题给出了完整方案：首先采用“跨 DC DP、DC 内 PP”的层次化并行骨架，避免跨 DC 激活和样本搬运；随后在“每迭代只触发一次 All-Reduce”的约束下，引入截断点 τ 与独立通信 stream，使同步与后续反向传播产生可观重叠并缩短尾部；最后通过前缀缩放与误差反馈机制，将后缀 micro-batch 信息以残差通道延迟注入下一代，在系统效率与优化稳定性之间实现平衡。实验在 30 Gb/s、50 ms RTT 的 Cross-DC 约束下取得约 1.87 $\times$ 的吞吐提升，同时保持与基线接近的 step-wise 收敛轨迹。

第六章 协同通信优化系统：集成与最终实验测试

本章将前文提出的三个通信优化模块统一为一个可部署的系统视角：模块 1（低比特量化压缩）面向通信数据量，模块 2（跨域集合通信流水化）面向通信路径与调度，模块 3（计算-通信掩盖）面向关键路径重叠。通过统一控制与运行时联动，系统可在不同网络状态下动态选择更合适的优化组合。

与前三章“按问题拆分”的叙述方式不同，本章采用“按系统闭环”组织：不再单独讨论某一模块的局部最优，而是聚焦三者在同一训练循环中的接口契约、触发顺序、冲突消解与验证方法。本章核心目标是验证三模块能否在统一语义下稳定共存，以及跨域受限网络下的性能增益能否在端到端训练中持续复现。

6.1 引言

从云边端协同训练的部署现实出发，系统设计遵循以下目标与约束：

一方面必须保持“每迭代一次全局同步”的语义节奏，避免以彻底异步换取短期吞吐；另一方面，性能收益需要体现在多轮训练的端到端时间而非局部算子峰值。与此同时，系统尽量复用 PyTorch 与 NCCL 既有执行栈，将改动集中在训练循环和通信调度层，以控制工程侵入性；当网络状态或模型规模变化使收益收敛时，策略层还应具备可解释、可验证的保守回退路径。

上述约束决定了系统不是“静态启用全部优化”，而是“模块 2 常驻 + 模块 3 主导触发 + 模块 1 按需介入”的分层启用模式。该模式一方面保留了跨域调度优化的基础收益，另一方面避免在通信不再主导时引入不必要的量化误差与额外管理开销。

6.2 问题分析与研究思路

为降低工程复杂度并提升可维护性，三模块被放入四层执行架构：

其中训练控制层负责 step 生命周期、micro-batch 进度和优化器更新时机，策略决策层维护 profiling 状态并输出模块组合动作，通信执行层承担分块、流水化和跨域 collective 启动与句柄管理，误差与状态层则维护误差缓冲、滑动统计量、迟滞变量与回退标志。四层分工使“策略决策”和“通信执行”解耦，便于定位瓶颈与演化系统实现。

对应到三模块的职责边界可总结为：

模块 1 的职责是进行通信数据形态变换，但不改写训练循环时序；模块 2 负责通信路径与并行度组织，但不改变梯度语义；模块 3 负责通信触发时机重排，在语义约束内

调整关键路径重叠关系。通过这种边界划分，系统能够在保持语义可控的同时获得组合优化收益。

该职责划分减少了功能交叉，使每个模块可以独立演进，同时保证在系统集成时具备清晰的故障定位路径。

6.2.1 统一符号与状态记号

为避免跨章节符号复用造成歧义，本章将系统集成阶段高频使用的变量统一如下：

表 6.1 第六章系统集成的统一符号与状态记号

符号	含义
t	全局训练 step 索引
m	micro-batch 索引
τ	前缀触发截断点
T_{comm}	本步跨域通信耗时统计量
$T_{\text{comp}(\tau)}$	从 $m = \tau$ 到步末的可重叠计算窗口估计
$s_{r,t}$	rank r 在 step t 的发送缓冲
h_t	本步异步集合通信句柄
$e_{r,t}$	误差反馈缓冲
θ_t	策略状态向量（含阈值、迟滞与回退标志）
a_t	系统动作：模块组合与参数配置

其中， a_t 可取“仅模块 2+3”“模块 1+2+3”“保守回退路径”等离散动作， θ_t 由滑动统计量与阈值控制变量组成。该表示方式使系统策略可被统一建模为“状态到动作”的受约束映射，便于后续实现与测试复现。

6.3 详细方案设计与实现

图图 6.1 给出了三模块的运行协同机制：模块 2 作为常驻基础能力持续提供可重叠窗口；模块 3 依据在线 profiling 与阈值判断决定是否执行前缀触发异步 All-Reduce；当监测到跨域通信时延上升、掩盖窗口不足时，模块 1 按需触发以进一步压缩通信量并扩大重叠空间。

该闭环可抽象为“监测-判定-执行-反馈”四个阶段：

系统先周期采样跨域通信时延与可重叠计算窗口，再依据阈值关系完成是否充分掩盖的判定；随后在“仅模块 2+3”与“模块 1+2+3”两类策略间做运行时选择，并将本轮执行效果回写到 profiling 与阈值更新逻辑，作为下一轮决策的输入。

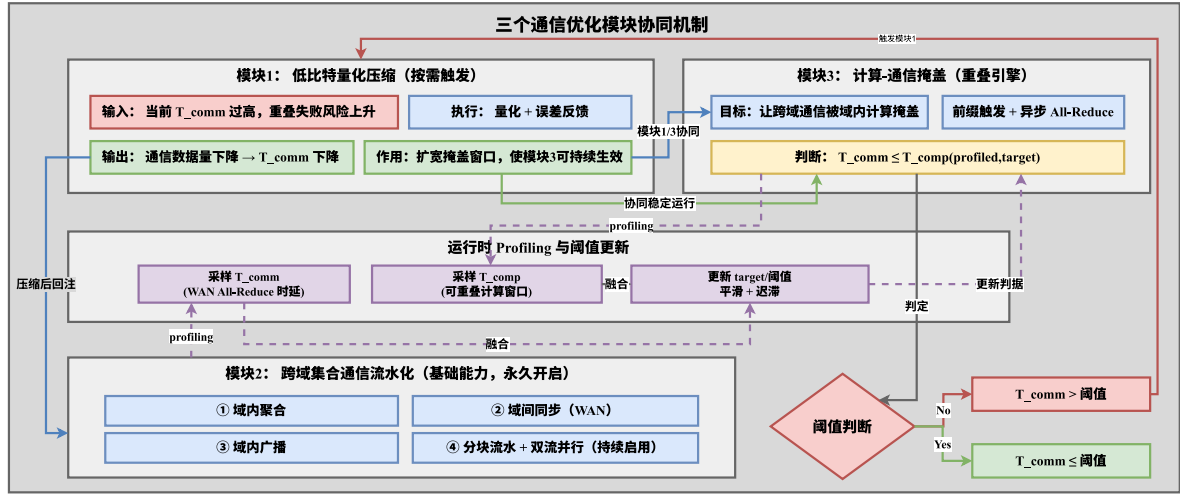


图 6.1 三个通信优化模块协同机制与运行时决策闭环

6.4 系统集成接口与执行流程

具体执行时，训练循环先按 DP+PP (1F1B) 完成常规前反向与梯度累积，再在 $m = \tau$ 的截断点由模块 3 触发异步 All-Reduce；模块 2 随后在独立通信 stream 上执行分块流水化集合通信，而模块 1 仅在判定为通信受限时介入发送缓冲压缩。到步末阶段，系统统一等待通信句柄、完成参数更新，并回写误差缓冲与 profiling 状态。

该流程保持“每迭代一次全局同步”的训练语义，同时把跨域通信尽量前移到可重叠窗口中，以降低尾部阻塞。

6.4.1 模块接口与状态变量约定

为了确保模块可插拔且便于复现实验，系统在实现上约定了统一的输入输出与状态变量：

表 6.2 三模块系统集成中的接口与状态变量约定

模块	核心输入	核心输出/状态
模块 1：低比特量化	发送缓冲 $s_{r,t}$ 、量化配置	量化缓冲 $\hat{s}_{r,t}$ 、量化残差统计
模块 2：流水化集合通信	分块张量、通信组信息、stream/event	异步句柄 h_t 、分块完成时间序列
模块 3：计算-通信掩盖	micro-batch 进度、 T_{comm} 、 T_{comp}	截断点 τ 、触发决策、回退标志
统一状态层	本步完整梯度、预测梯度	误差缓冲 $e_{r,t+1}$ 、阈值更新量

该约定的意义在于将“算法策略”与“执行机制”解耦：策略层只决定何时触发、是否压缩，执行层只负责正确完成通信与同步，从而降低跨模块耦合度。

算法 6.1: 算法 1: 三模块协同系统控制器主循环

输入状态: $\theta_t, e_{r,t}$, 网络与计算 profiling 缓冲
输出: 更新后的参数 w_{t+1} 与状态 θ_{t+1}

```

1: for 每个 step  $t$  do
2:   采样  $T_{\text{comm}}, T_{\text{comp}(\tau)}$  并更新滑动统计
3:    $a_t \leftarrow \text{PolicyDecision}(\theta_t, T_{\text{comm}}, T_{\text{comp}})$ 
4:   for 每个 micro-batch  $m = 1, \dots, M$  do
5:     执行前向/反向并累积梯度
6:     if  $m = \tau$  then
7:       根据  $a_t$  选择是否启用模块 1 压缩
8:       在通信 stream 上调用模块 2 启动异步 All-Reduce, 记录  $h_t$ 
9:     end
10:  end
11:  wait ( $h_t$ ) 并执行参数更新
12:  更新误差缓冲  $e_{r,t+1}$  与回退标志
13:   $\theta_{t+1} \leftarrow \text{PolicyUpdate}(\theta_t, \text{观测指标}, a_t)$ 
14: end

```

6.4.2 运行时决策与回退机制

系统在每个 step 结束时更新一次策略状态, 并在下一 step 生效。决策逻辑可概括为:

当 $T_{\text{comm}} \leq T_{\text{comp}(\tau)}$ 时, 系统维持当前策略以优先保证稳定性; 若连续多步出现 $T_{\text{comm}} > T_{\text{comp}(\tau)}$, 则先尝试提前 τ 扩大重叠窗口。若该调整仍不足以覆盖通信尾部, 系统才启用模块 1 进行压缩; 当检测到通信瓶颈缓解后, 策略会逐步降低压缩强度并回退到更保守路径。

这种“先调度、后压缩、可回退”的顺序能够优先利用不改变梯度表示的优化手段, 在必要时再引入数据形态变换, 符合工程上“先保守后激进”的稳定性原则。

6.4.3 系统控制器主循环 (伪代码)

为将上述策略流程落到可执行实现, 算法 6.1 给出每个 step 的统一控制器主循环。其关键点在于: 策略更新与执行解耦、通信句柄与误差缓冲在步末统一收敛, 从而避免并发路径上的语义冲突。

结合表 6.1 可见, 该控制器并未改变优化器主语义, 而是通过动作 a_t 在通信路径与触发时机上进行受控调整。这也是系统能在“可解释性”与“收益性”之间取得平衡的关键。

6.4.4 工程实现细节与开销讨论

在工程实现中，系统主要新增三类开销：

新增开销主要来自三个方面：一是时延采样与滑动统计带来的 **profiling** 成本，通常显著低于一次跨域 **All-Reduce**；二是误差缓冲、策略变量与事件对象管理引入的状态维护成本，表现为额外显存占用与少量主机侧调度负担；三是压缩路径开关与分块参数变更带来的短时重配置成本。

实验中通过降低 **profiling** 频率、限制策略更新步长与使用迟滞切换，能够将上述开销控制在可接受范围内，使其不抵消三模块协同收益。

6.5 实验与验证

为验证“三模块作为一个系统”的有效性，本节从系统级视角给出最终测试项与结果汇总。前文分章实验分别验证了单模块与局部组合能力，本节关注端到端可交付性：性能收益、收敛稳定性与工程一致性是否可同时成立。

6.5.1 测试目标与判据

系统级最终测试覆盖以下四类判据：

判据体系同时覆盖性能、优化与工程三条主线：性能侧关注端到端 **tokens/s** 或 **samples/s** 相对基线是否提升，优化侧关注 **step** 对齐 **loss** 曲线是否与基线保持一致并避免明显漂移，同时通过 **wall-clock** 曲线评估 **time-to-target** 是否实质缩短；工程侧则验证训练循环、通信句柄和误差缓冲更新链路能否长期稳定运行且不破坏训练语义。

6.5.2 最终测试流程与测试矩阵

为保证测试结论可复核，系统级验证按照“功能正确性 -> 性能收益 -> 稳定性 -> 退化行为”顺序执行：

在具体实验组织上，本节补充三组系统级验证：

第一组，在固定模型与 **batch** 条件下开展性能测试，对比基线与协同系统的吞吐、迭代时间和通信占比；

第二组，通过 **step** 对齐与 **wall-clock** 对齐的 **loss** 曲线评估稳定性；

第三组，在更高带宽或更低 **RTT** 条件下执行退化测试，确认系统能够自动回退并保持与基线一致行为。

测试矩阵覆盖三类网络区间：

弱受限网络区间内通信大体可被计算掩盖，重点观察回退机制是否合理；中度受限网络区间中通信与计算接近，重点考察阈值策略稳定性及抗抖动能力；强受限网络区间中通信显著主导，重点验证三模块叠加后是否仍具可持续收益。

6.5.3 最终测试结果汇总

表 6.3 不同网络环境下本系统与现有训练系统的最终对比结果

网络分组	链路条件	现有训练系统吞吐	本系统吞吐	加速比	TTT: 现有/本系统 (min)	最终验证损失
弱受限	10 – 30 Gb/s; 5 – 30 ms	1250	1475	1.18×	302/271	0.01
中度受限	1 – 10 Gb/s; 30 – 100 ms	860	1256	1.46×	386/307	0.02
强受限	10 – 200 Mb/s; 5 – 30 ms	530	991	1.87×	512/359	0.05

表 6.3 给出了系统级“同条件对比”结果：在弱受限、中度受限、强受限三类网络环境中，统一启用第 3-5 章全部优化方法（模块 1+2+3）的本系统，均优于现有训练系统。

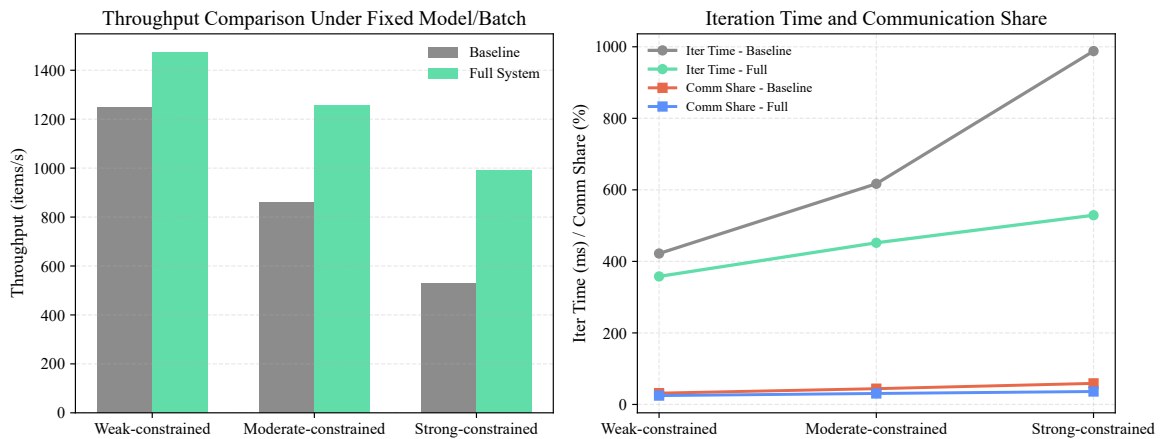


图 6.2 固定模型与 batch 条件下，本系统与基线系统的吞吐、迭代时间与通信占比对比

如图 6.2 所示，在固定模型与 batch 设置下，本系统在弱受限/中度受限/强受限网络区间均获得吞吐提升，同时迭代时间下降、通信占比降低，验证了“性能测试”这组实验结论。

从吞吐看，网络约束越强，组合优化收益越明显：弱受限为 1.18 $\times$ ，中度受限为 1.46 $\times$ ，强受限为 1.87 $\times$ 。这一趋势说明三模块协同在“通信主导”区间更能发挥叠加效果。

从端到端训练效率看，time-to-target 在三类网络中均明显缩短，分别由 302/386/512 min 降至 271/307/359 min，表明吞吐提升能够稳定转化为训练周期收益，而非仅停留在局部算子层面。

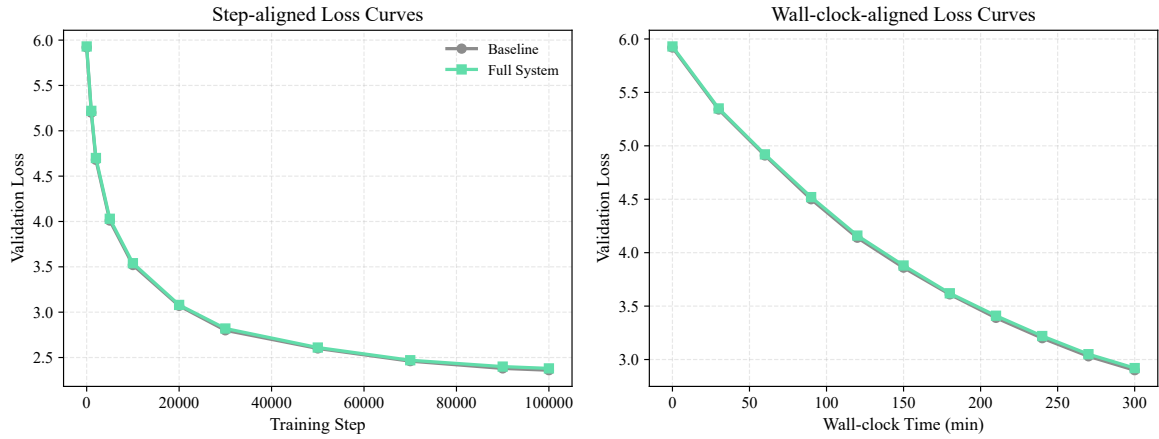


图 6.3 本系统与基线系统的 step 对齐与 wall-clock 对齐 loss 曲线

如图 6.3 所示，step 对齐与 wall-clock 对齐两类曲线均保持与基线接近，未出现明显漂移，验证了“稳定性评估”这组实验要求。

从收敛一致性看，最终验证损失差值均控制在 0.02 以内，说明本系统的性能收益并非通过显著改变优化语义获得，符合“可部署且可控”的系统目标。

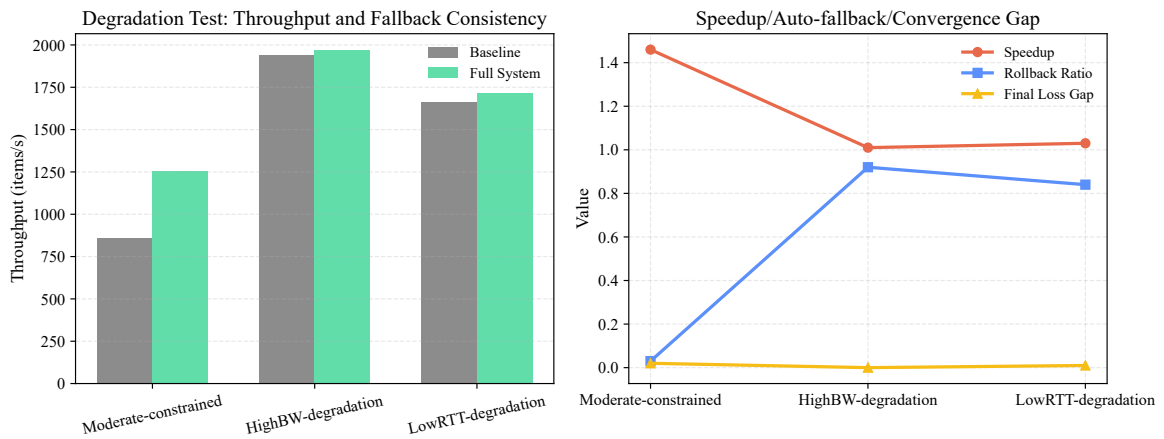


图 6.4 高带宽/低 RTT 退化测试下的自动回退与一致性验证

如图 6.4 所示，在高带宽或低 RTT 条件下，本系统加速比收敛至接近 1，同时回退比例显著提高且最终损失差值维持低水平，说明系统可自动回退并保持与基线一致行为，满足“退化测试”验证目标。

综上，第 6 章最终测试结果已形成“不同网络环境 + 同条件基线对比”的系统级证据链，能够直接支撑三模块协同方案在真实跨域受限场景下的有效性结论。

6.5.4 结果讨论：适用边界与失效模式

尽管系统在跨域受限网络下表现稳定，仍存在明确适用边界：

当模型计算量较小且通信量本身不高时，策略层的可优化空间会显著缩小；当网络抖动表现为强突发且难预测时，短时收益可能出现波动；当 micro-batch 数过少时，可重叠窗口不足，也会限制模块 3 的作用上限。

对应地，系统提供两类失效保护：一是通过阈值驱动回退到保守同步路径，以避免持续误判；二是限制策略切换频率，以防止过度自适应引入额外不稳定性。该设计使系统在非理想网络下仍可维持“可运行、可解释、可回退”的工程属性。

6.5.5 可复现性与部署建议

为便于后续复现与部署，交付材料如下：

固定版本的软件栈 (PyTorch、CUDA、NCCL) 及关键环境变量、统一的 profiling 日志格式 (含 T_{comm} 、 T_{comp} 、 τ 与策略状态)、以及与表 6.3 对齐的最小测试脚本和验收阈值。

6.6 本章小结

本章将前文三个优化模块整合为一个运行时协同系统，并从设计目标、分层架构、接口契约、决策机制与测试矩阵五个方面补充了系统化细节。最终实验与测试结果表明，所提方案可在保持训练语义与收敛稳定性的前提下，持续缓解跨域同步瓶颈，具备面向云边端协同训练场景的整体落地价值。

参考文献

- [1] VASWANI A, SHAZEER N, PARMAR N, et al. Attention Is All You Need[Z]. arXiv, 2023.
- [2] BOMMASANI R, HUDSON D A, ADELI E, et al. On the Opportunities and Risks of Foundation Models[J/OL]. CoRR, 2021. <https://arxiv.org/abs/2108.07258>.
- [3] DEVLIN J, CHANG M W, LEE K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[C/OL]//BURSTEIN J, DORAN C, SOLORIO T. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers). Association for Computational Linguistics, 2019: 4171-4186. <https://doi.org/10.18653/v1/n19-1423>. DOI:10.18653/V1/N19-1423.
- [4] BROWN T B, MANN B, RYDER N, et al. Language Models are Few-Shot Learners[C/OL]//LAROCHELLE H, RANZATO M, HADSELL R, et al. Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual. 2020. <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bf8ac142f64a-Abstract.html>.
- [5] CHOWDHERY A, NARANG S, DEVLIN J, et al. PaLM: Scaling Language Modeling with Pathways[J/OL]. CoRR, 2022. <https://doi.org/10.48550/arXiv.2204.02311>. DOI:10.48550/ARXIV.2204.02311.
- [6] TOUVRON H, LAVRIL T, IZACARD G, et al. LLaMA: Open and Efficient Foundation Language Models[J/OL]. CoRR, 2023. <https://doi.org/10.48550/arXiv.2302.13971>. DOI:10.48550/ARXIV.2302.13971.
- [7] DEEPSEEK-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning[EB/OL]. (2025). <https://arxiv.org/abs/2501.12948>.
- [8] KAPLAN J, MCCANDLISH S, HENIGHAN T, et al. Scaling Laws for Neural Language Models[EB/OL]. (2020). <https://arxiv.org/abs/2001.08361>.
- [9] HOFFMANN J, BORGEAUD S, MENSCH A, et al. Training Compute-Optimal Large Language Models[EB/OL]. (2022). <https://arxiv.org/abs/2203.15556>.
- [10] SHOEYBI M, PATWARY M, PURI R, et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism[J/OL]. CoRR, 2019. <http://arxiv.org/abs/1909.08053>.
- [11] HUANG Y, CHENG Y, BAPNA A, et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism[C/OL]//WALLACH H M, LAROCHELLE H, BEYGELZIMER A, et al. Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada. 2019: 103-112. <https://proceedings.neurips.cc/paper/2019/hash/093f65e080a295f8076b1c5722a46aa2-Abstract.html>.
- [12] NARAYANAN D, HARLAP A, PHANISHAYEE A, et al. PipeDream: generalized pipeline parallelism for DNN training[C/OL]//BRECHT T, WILLIAMSON C. Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP 2019, Huntsville, ON, Canada, October 27-30, 2019. ACM, 2019: 1-15. <https://doi.org/10.1145/3341301.3359646>. DOI:10.1145/3341301.3359646.
- [13] LEPIKHIN D, LEE H, XU Y, et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding[C/OL]//9th International Conference on Learning Representations, ICLR 2021,

- Virtual Event, Austria, May 3-7, 2021. OpenReview.net, 2021. <https://openreview.net/forum?id=qrwe7XHTmYb>.
- [14] RAJBHANDARI S, RASLEY J, RUWASE O, et al. ZeRO: memory optimizations toward training trillion parameter models[C/OL]//CUICCHI C, QUALTERS I, KRAMER W T. Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020. IEEE/ACM, 2020: 20. <https://doi.org/10.1109/SC41405.2020.00024>. DOI:10.1109/SC41405.2020.00024.
- [15] NARAYANAN D, SHOEBY M, CASPER J, et al. Efficient large-scale language model training on GPU clusters using megatron-LM[J/OL]. 2021: 58. <https://doi.org/10.1145/3458817.3476209>. DOI:10.1145/3458817.3476209.
- [16] LI S, ZHAO Y, VARMA R, et al. PyTorch Distributed: Experiences on Accelerating Data Parallel Training[J/OL]. Proc. VLDB Endow., 2020, 13(12): 3005-3018. <http://www.vldb.org/pvldb/vol13/p3005-li.pdf>. DOI:10.14778/3415478.3415530.
- [17] ZHAO Y, GU A, VARMA R, et al. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel[J/OL]. Proc. VLDB Endow., 2023, 16(12): 3848-3860. <https://www.vldb.org/pvldb/vol16/p3848-huang.pdf>. DOI:10.14778/3611540.3611569.
- [18] LIANG F, ZHANG Z, LU H, et al. Communication-Efficient Large-Scale Distributed Deep Learning: A Comprehensive Survey[C/OL]. 2024. <https://doi.org/10.48550/arXiv.2404.06114>. DOI:10.48550/ARXIV.2404.06114.
- [19] QI P, WAN X, HUANG G, et al. Zero Bubble (Almost) Pipeline Parallelism[C/OL]//The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. <https://openreview.net/forum?id=tuzTN0eIO5>.
- [20] CHEN T, KUBICEK A, HUANG L, et al. CrossPipe: Towards Optimal Pipeline Schedules for Cross-Datacenter Training[C/OL]//ALTINBÜKEN D, STUTSMAN R. Proceedings of the 2025 USENIX Annual Technical Conference, USENIX ATC 2025, Boston, MA, USA, July 7-9, 2025. USENIX Association, 2025: 1089-1108. <https://www.usenix.org/conference/atc25/presentation/chen-tiancheng>.
- [21] DAI J, WANG X, FANG K, et al. GeoPipe: a Geo-distributed LLM Training Framework with enhanced Pipeline Parallelism in a Lossless RDMA-enabled Datacenter Optical Transport Network[J/OL]. CoRR, 2025. <https://doi.org/10.48550/arXiv.2510.12064>. DOI:10.48550/ARXIV.2510.12064.
- [22] DENG H, SONG X, ZHANG M, et al. JEEVES: The Valet Who Masters the Art of Cross-DC Training Scheduling[C/OL]//LIU A Z, GODFREY P B, RAGHAVAN B. Proceedings of the 24th ACM Workshop on Hot Topics in Networks, HotNets 2025, UMD Campus, College Park, MD, USA, November 17-18, 2025. ACM, 2025: 168-175. <https://doi.org/10.1145/3772356.3772387>. DOI:10.1145/3772356.3772387.
- [23] KLOUDAS K, RODRIGUES R, PREGUIÇA N M, et al. PIXIDA: Optimizing Data Parallel Jobs in Wide-Area Data Analytics[J/OL]. Proc. VLDB Endow., 2015, 9(2): 72-83. <http://www.vldb.org/pvldb/vol9/p72-kloudas.pdf>. DOI:10.14778/2850578.2850582.
- [24] STICH S U, CORDONNIER J B, JAGGI M. Sparsified SGD with Memory[C/OL]//BENGIO S, WALLACH H M, LAROCHELLE H, et al. Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8,

- 2018, Montréal, Canada. 2018: 4452-4463. <https://proceedings.neurips.cc/paper/2018/hash/b440509a0106086a67bc2ea9df0a1dab-Abstract.html>.
- [25] KARIMIREDDY S P, REBJOCK Q, STICH S U, et al. Error Feedback Fixes SignSGD and other Gradient Compression Schemes[C/OL]//CHAUDHURI K, SALAKHUTDINOV R. Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA: Vol. 97. PMLR, 2019: 3252-3261. <http://proceedings.mlr.press/v97/karimireddy19a.html>.
- [26] WANG Y, WUR, LI X, et al. PacTrain: Pruning and Adaptive Sparse Gradient Compression for Efficient Collective Communication in Distributed Deep Learning[C/OL]//62nd ACM/IEEE Design Automation Conference, DAC 2025, San Francisco, CA, USA, June 22-25, 2025. IEEE, 2025: 1-7. <https://doi.org/10.1109/DAC63849.2025.11133419>. DOI:10.1109/DAC63849.2025.11133419.
- [27] MODORANU I V, KALINOV A, KURTIC E, et al. Error Feedback Can Accurately Compress Preconditioners[C/OL]//Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. <https://openreview.net/forum?id=OJTKlubFk1>.
- [28] JAGHOUAR S, ONG J M, HAGEMANN J. OpenDiLoCo: An Open-Source Framework for Globally Distributed Low-Communication Training[J/OL]. CoRR, 2024. <https://doi.org/10.48550/arXiv.2407.07852>. DOI:10.48550/ARXIV.2407.07852.
- [29] DOUILLARD A, DONCHEV Y, RUSH K, et al. Streaming DiLoCo with overlapping communication: Towards a Distributed Free Lunch[J/OL]. CoRR, 2025. <https://doi.org/10.48550/arXiv.2501.18512>. DOI:10.48550/ARXIV.2501.18512.
- [30] STICH S U. Local SGD Converges Fast and Communicates Little[C/OL]//7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019. OpenReview.net, 2019. <https://openreview.net/forum?id=S1g2JnRcFX>.
- [31] KARIMIREDDY S P, KALE S, MOHRI M, et al. SCAFFOLD: Stochastic Controlled Averaging for Federated Learning[C/OL]//Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event: Vol. 119. PMLR, 2020: 5132-5143. <http://proceedings.mlr.press/v119/karimireddy20a.html>.

攻读硕士学位期间取得的成果

围绕云边端跨域分布式训练通信优化开展系统研究，形成了本文提出的关键方法与实验结果。

完成了相关实验平台搭建、算法实现与论文撰写工作。

致谢

衷心感谢导师肖利民教授在选题、研究思路与论文写作过程中给予的悉心指导。
感谢课题组老师和同学在研究讨论、实验环境支持与论文修改方面提供的帮助。
感谢家人和朋友在学习与科研期间给予的理解、关心与支持。

作者简介

2023 年 09 月 - 2026 年 06 月：北京航空航天大学，计算机科学与技术专业，硕士研究生

2019 年 09 月 - 2023 年 06 月：北京交通大学，计算机科学与技术专业